Object Detection

Jonghyun Choi

Associate Professor

Department of Electrical and Computer Engineering
Seoul National University

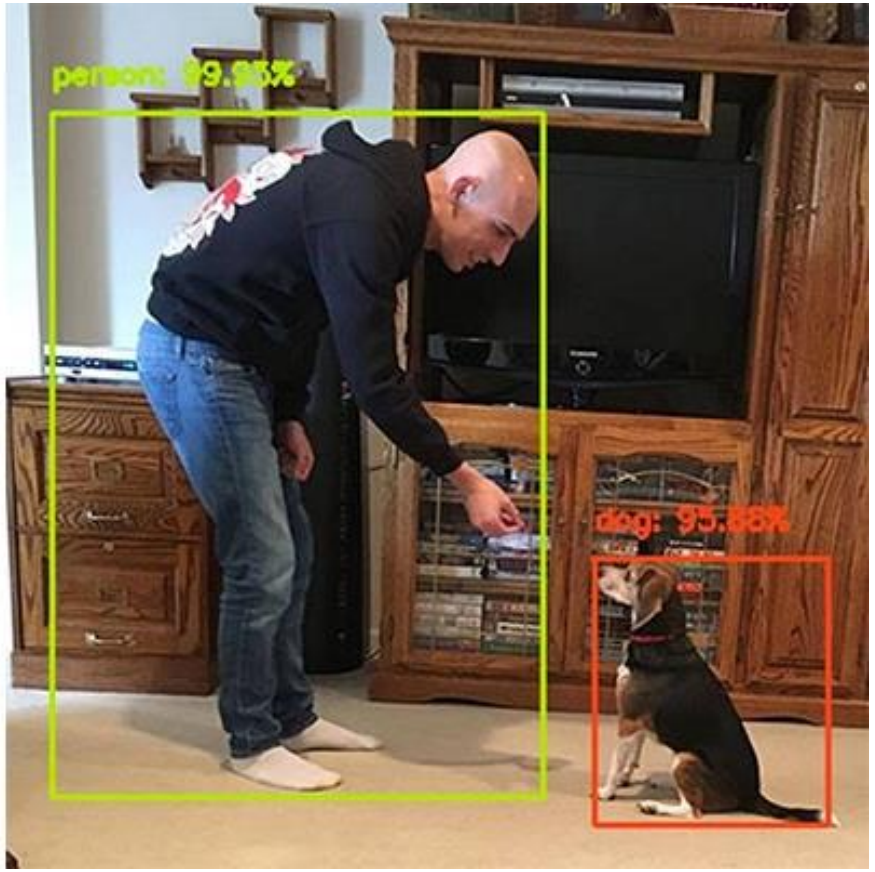
Schedule

Each hour is scheduled as **50 min lecture + 10 min break**

- **Day 1**: 4 hr lecture + 4 hr practice
  - **AM**: CNN, Transformers, Object detection
  - **PM**: DETR, MI-DETR
- **Day 2**: 4 hr lecture + 4 hr practice
  - **AM**: Self-supervised learning, Weakly supervised learning, Multimodal models, Semantic segmentation
  - **PM**: DINO, SAM variants

Image classification **vs** object detection

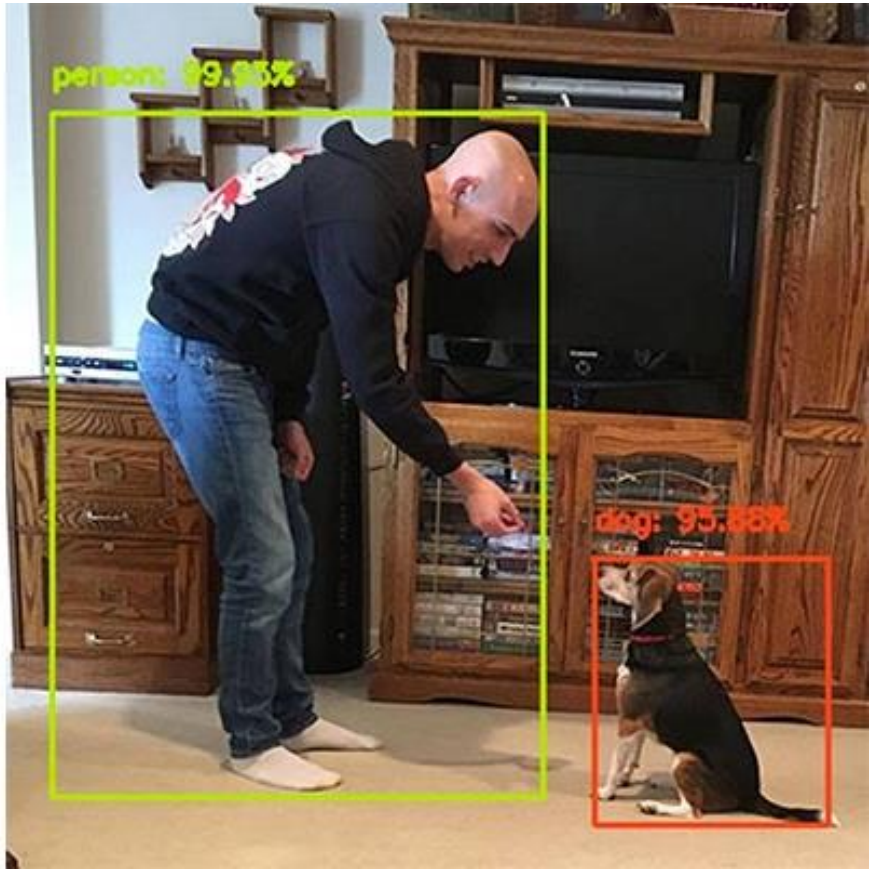
Image classification



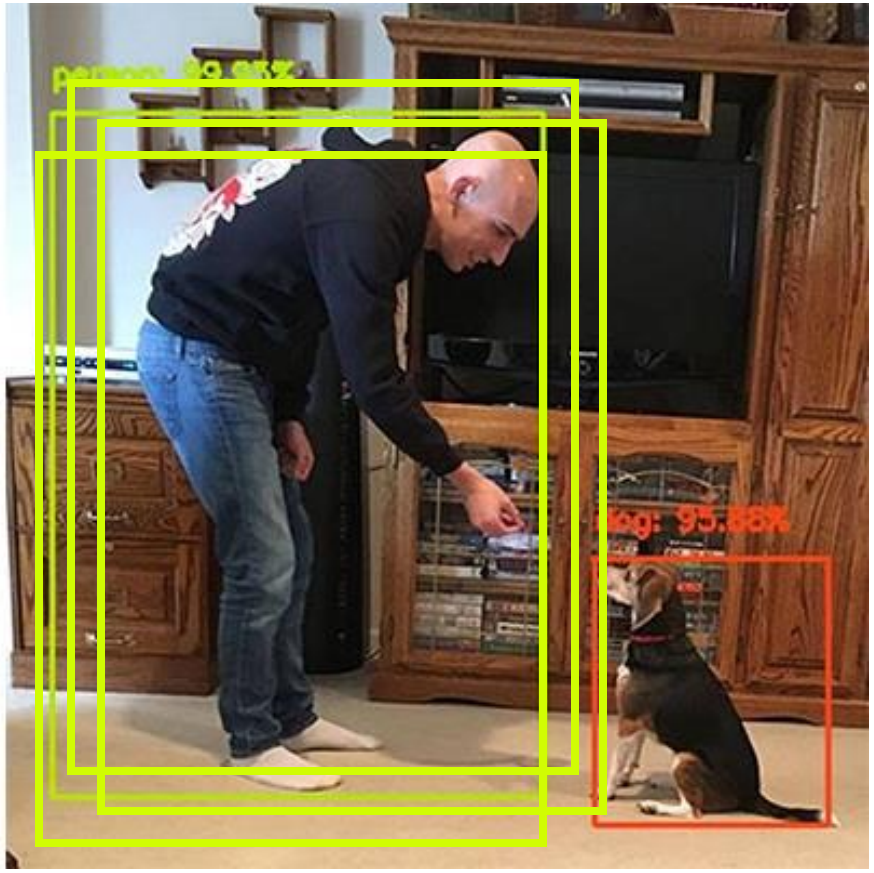
Object detection

Source: https://www.pyimagesearch.com/2018/05/14/a-gentle-guide-to-deep-learning-object-detection/

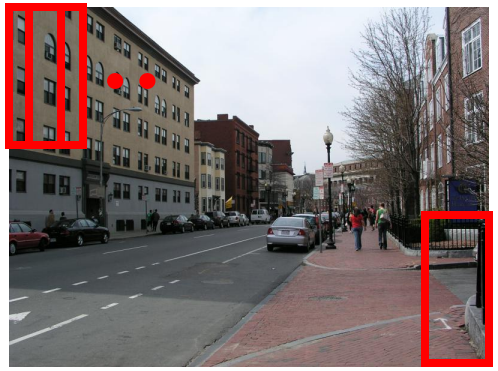
Are they solving different problem?



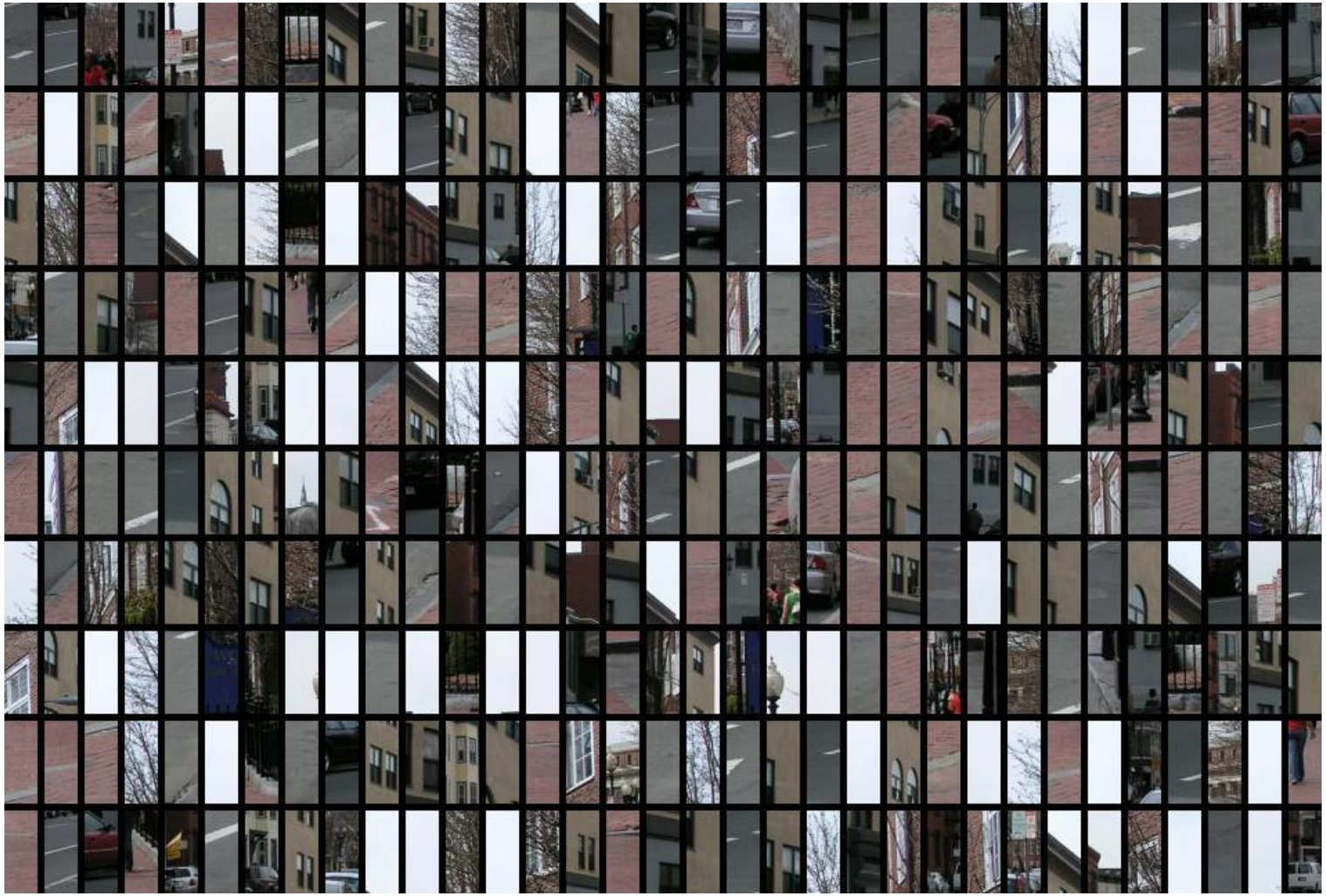
Are they solving same problem?



Sliding window: a simple alignment solution



Each window is separately classified



Evaluation - IOU

Multiple classes, multiple objects per images

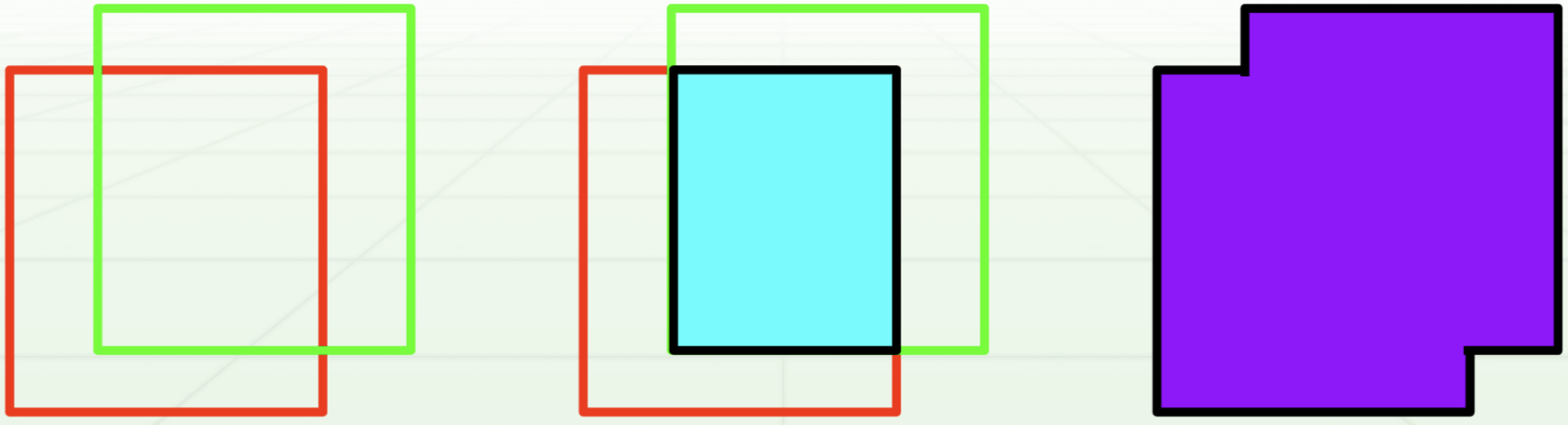
Can’t just use accuracy

“Correct” bounding box:

Intersection / Union > 0.5

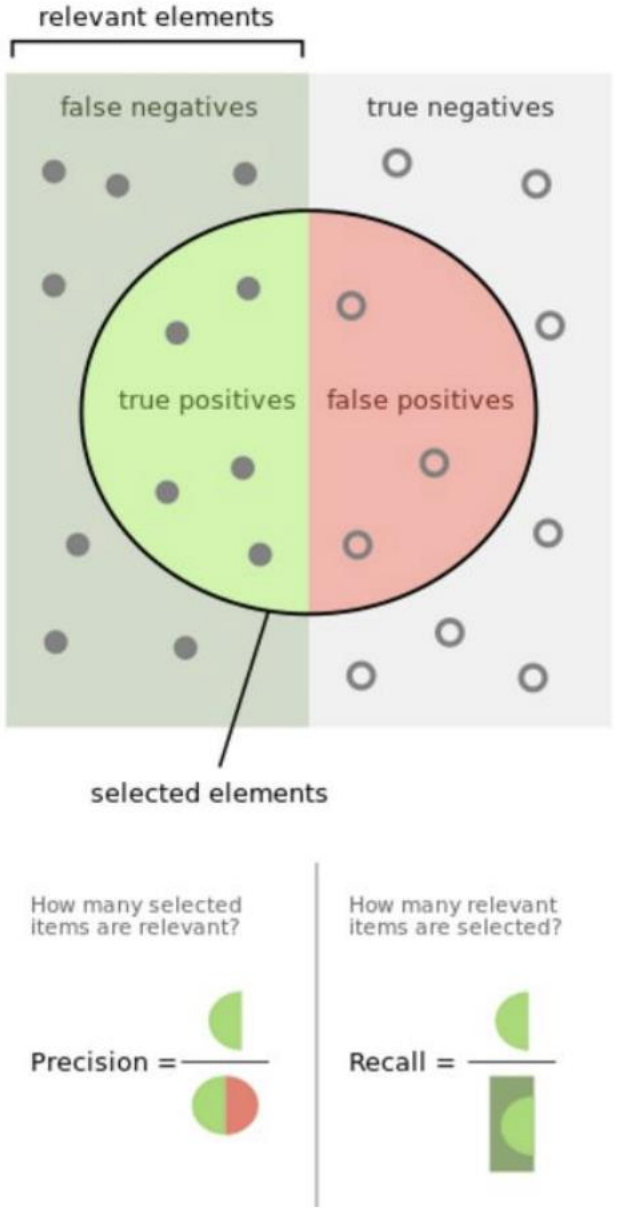
Intersection: Ground truth ∩ prediction

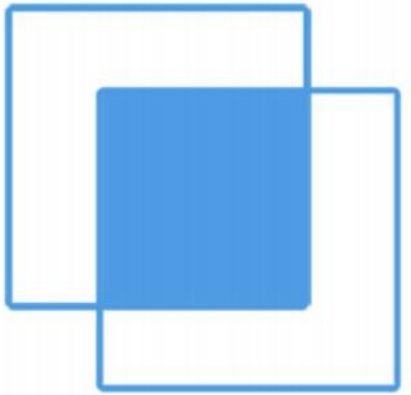
Union: Ground truth ∪ prediction

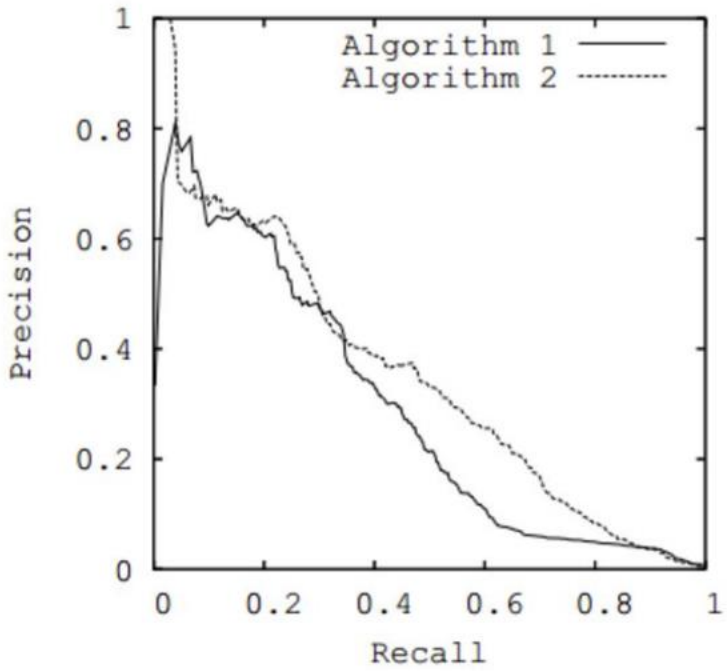


Slide by Joe Redmon

Precision and Recall







https://en.wikipedia.org/wiki/Precision_and_recall

Slide by Liang, Lu, Perkins

Evaluation – PR curve

"Correct" bounding box:

    Intersection / Union > 0.5

Recall:

    Correct bounding boxes / total ground-truth boxes

Precision:

    Correct bounding boxes / total predicted boxes

Only the most confident predictions:	High precision, low recall
All the predictions:	Low precision, high recall

Slide by Joe Redmon

Evaluation – PR curve

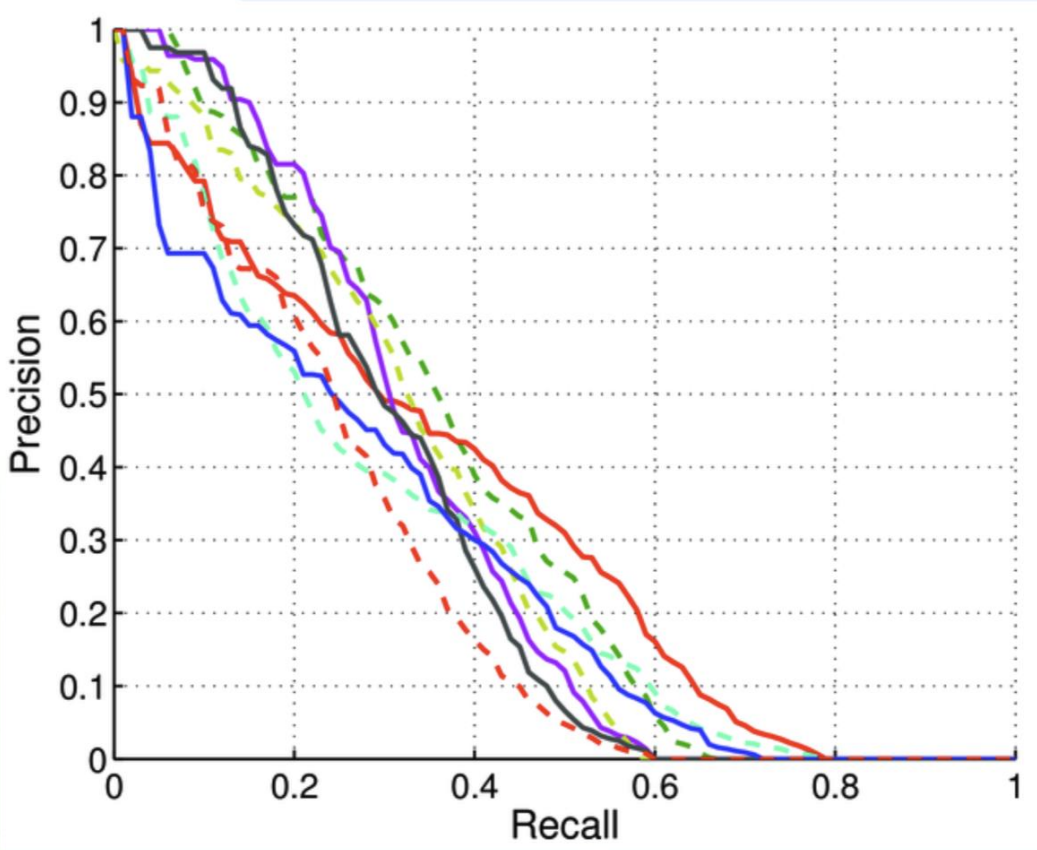
Precision-Recall curve: vary threshold, plot precision and recall

Average precision:

- Area under PR curve
- Only for a single class

Take mean of AP across classes:

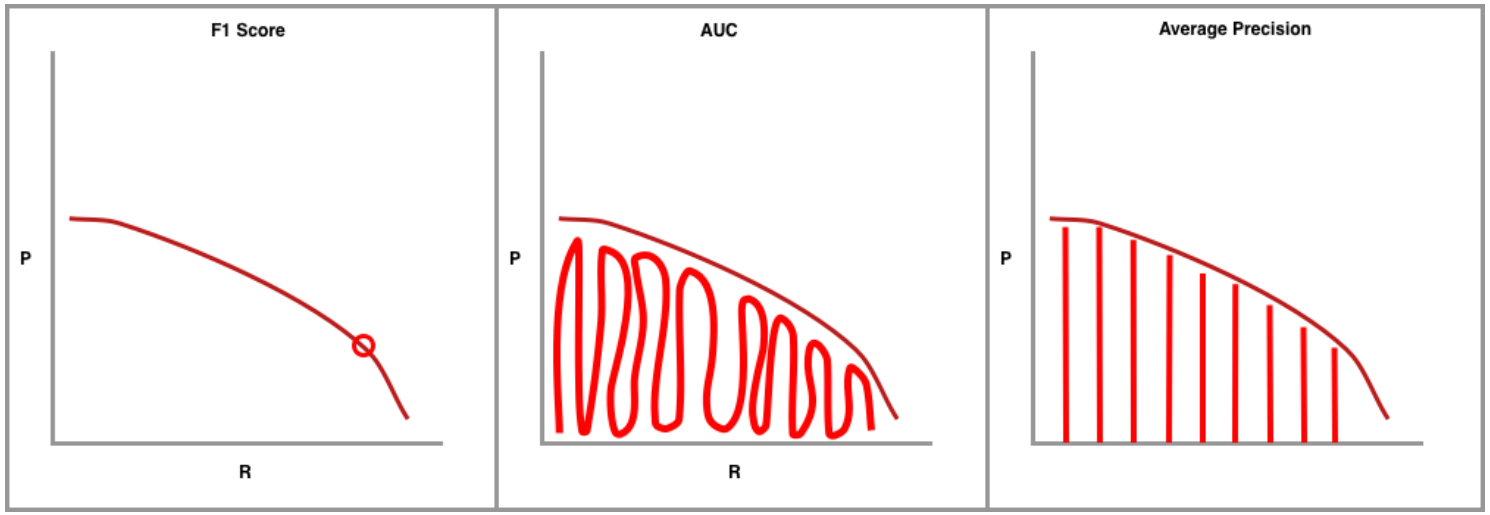
- Mean AP (mAP)
- Standard detection metric
- Sometimes at particular IOU
  - I.e. mAP@.5 or mAP@.75



Slide by Joe Redmon

F1 score **vs** AUC **vs** AP

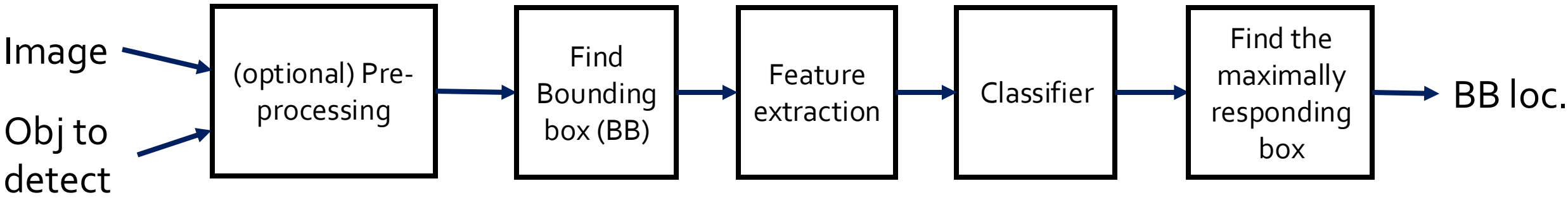
- **F1** = $2 \cdot \frac{P \cdot R}{P + R}$
- **AUC**: Area under the curve
- **AP**: Average precision (sparsely sampled version of AUC)
- **mAP**: mean Average Precision (mean of AP's over classes)



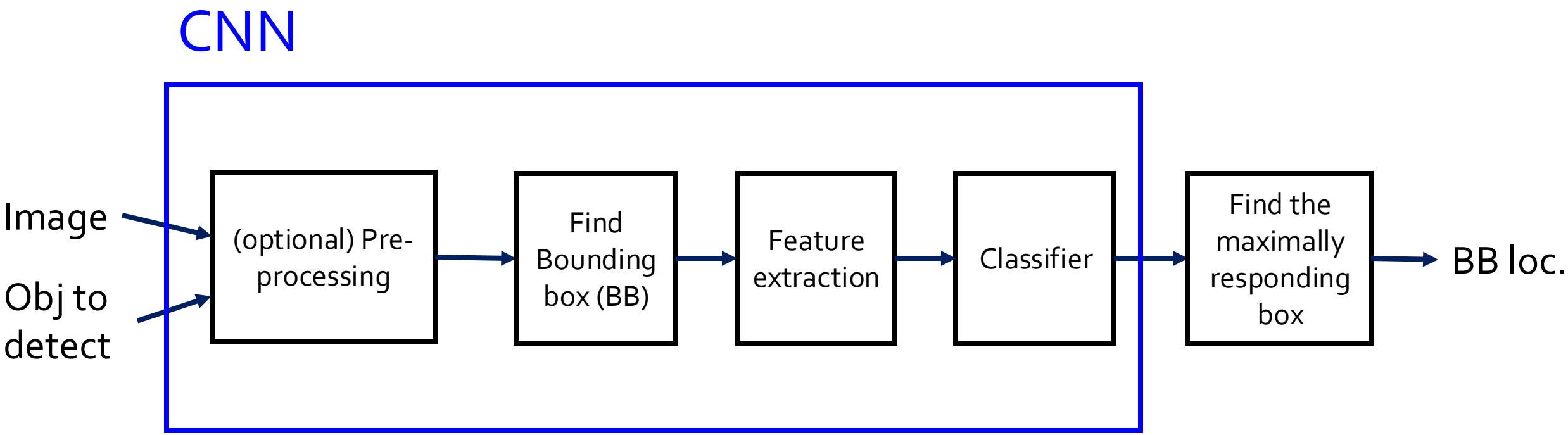
AP_{IOU}

- At a certain IOU value, we can compute AP
- AP_{50}
- AP_{70}
- AP: average of AP_{50} to AP_{95} with step of 5%
- AP_S: AP for small object (area < 32x32)
- AP_M: AP for medium sized object (32x32 < area < 96x96)
- AP_L: AP for large object (area > 96x96)

Historic pipeline

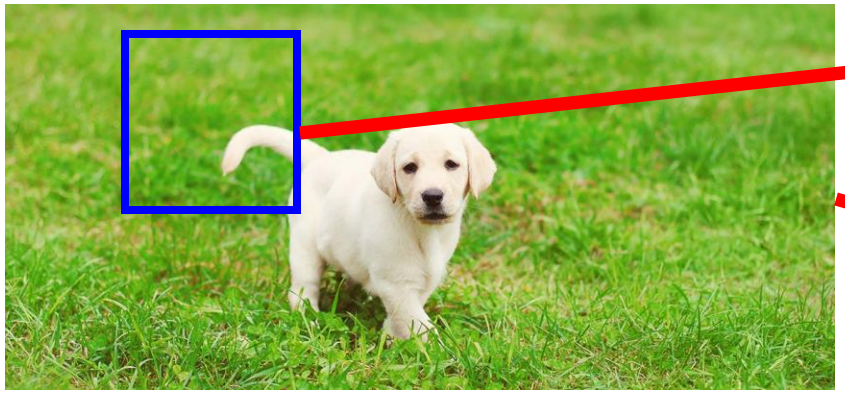


New pipeline



Using CNN in two ways

1. For each region, use CNN
→ **Two stage detection**



CNN

CNN

2. For the image as a whole, use CNN
→ **Single stage detection**

Using CNN in two ways

- Two stage detection
  - Slow
  - Accurate
  - E.g., R-CNN and its variants (Fast R-CNN, Faster R-CNN)
- Single stage detection
  - Fast
  - Less accurate (especially for the small objects)
  - E.g., YOLO, SSD

Two stage detection

- First object detection method using CNN
- Instead of using ‘sliding window’ use selective search to come up with regional proposal

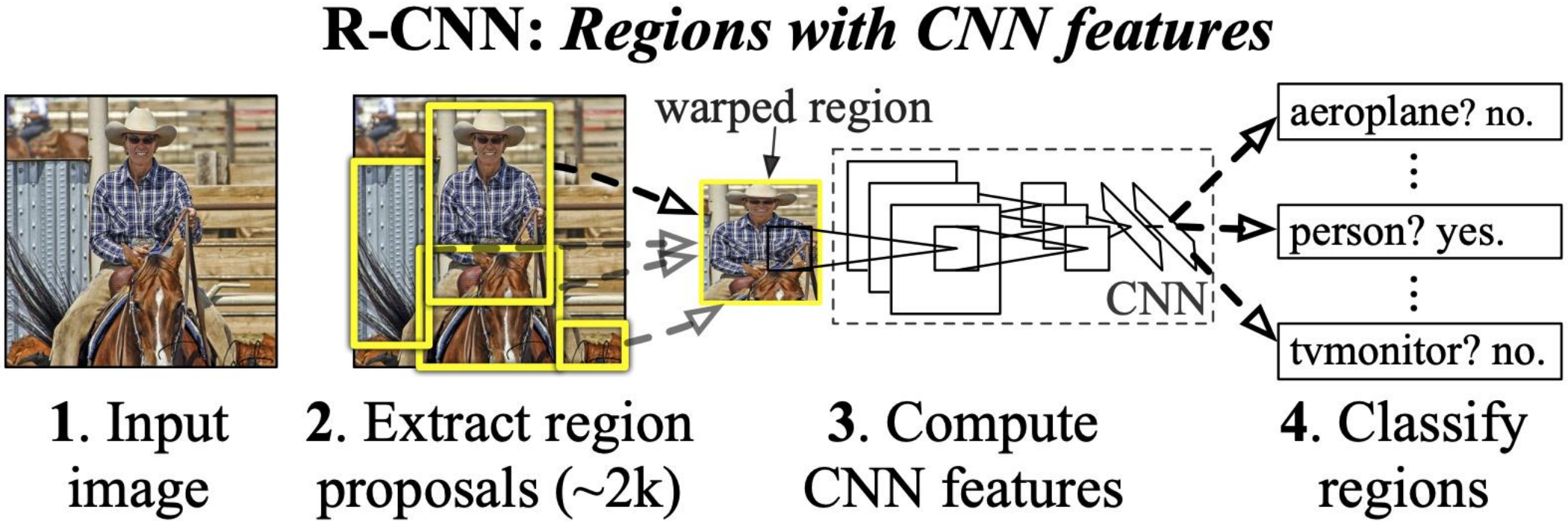
Girshick et al., “**Rich feature hierarchies for accurate object detection and semantic segmentation**” *CVPR 2014*

R-CNN, its variants and beyond

- R-CNN
- Fast R-CNN
- Faster R-CNN
- R-FCN

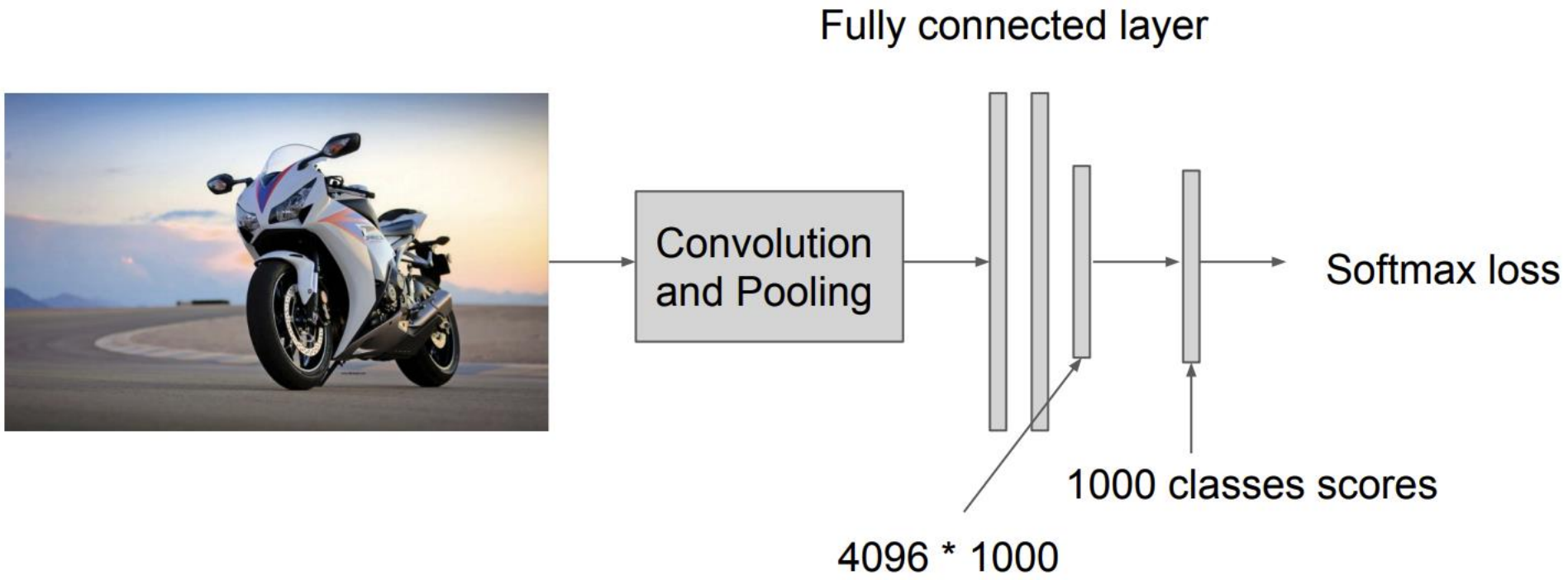
R-CNN

- Region CNN
- Not RNN (**Recurrent Neural Network** or **Recursive Neural Network**)



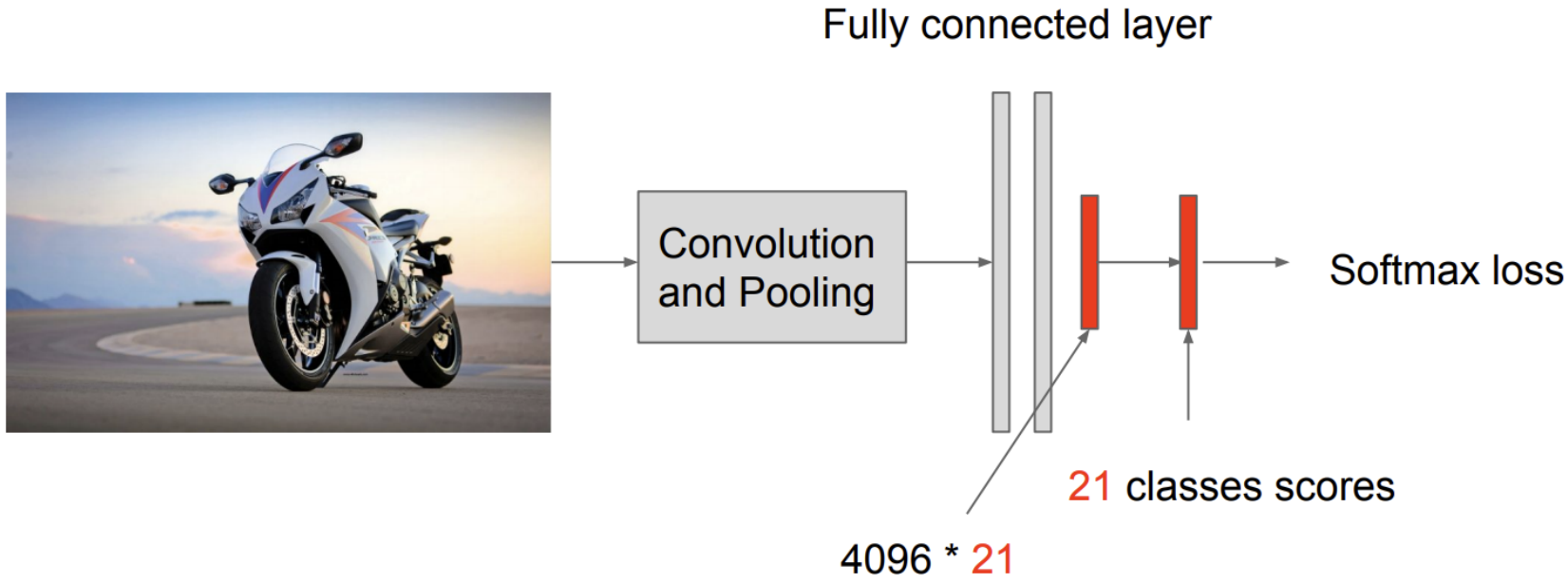
Training RCNN

- **Step 1**: train your own CNN model for classification (or use existing model) using ImageNet dataset.



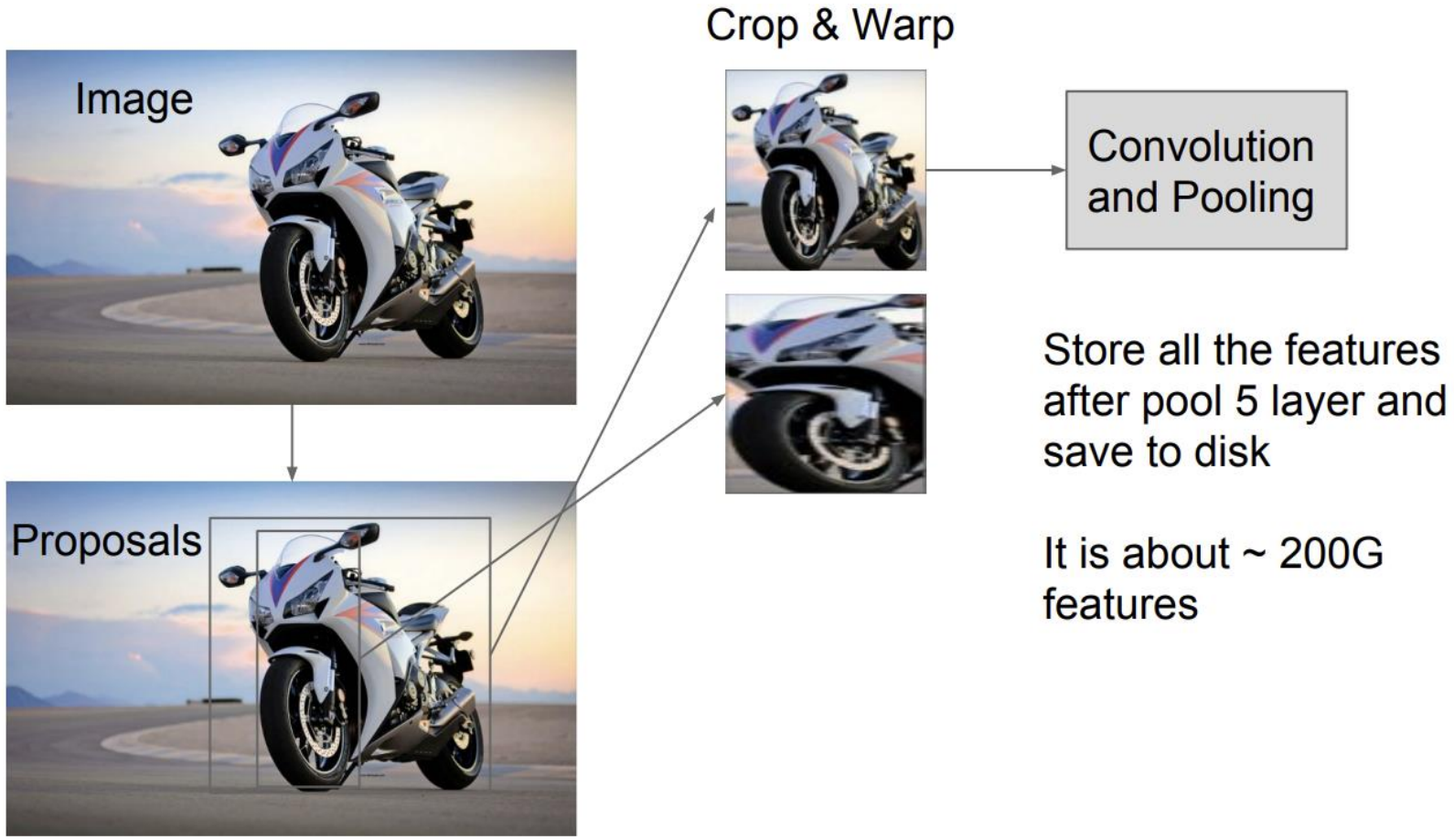
Training RCNN

- **Step 2**
  - Focus on 20 classes + 1 background
  - Remove the last FC layer and replace it with a smaller layer and fine-tune the model using PASCAL VOC dataset



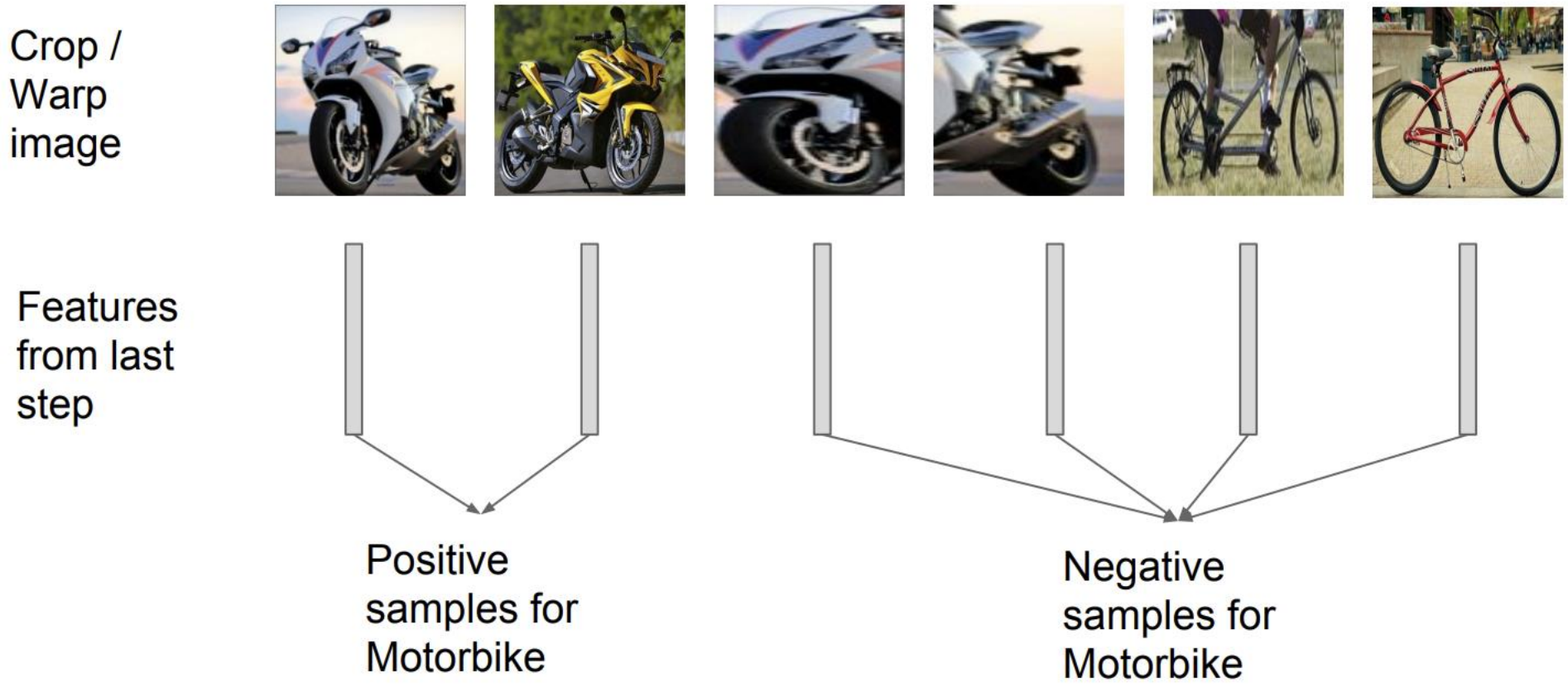
Training RCNN

- **Step 3**: extract feature



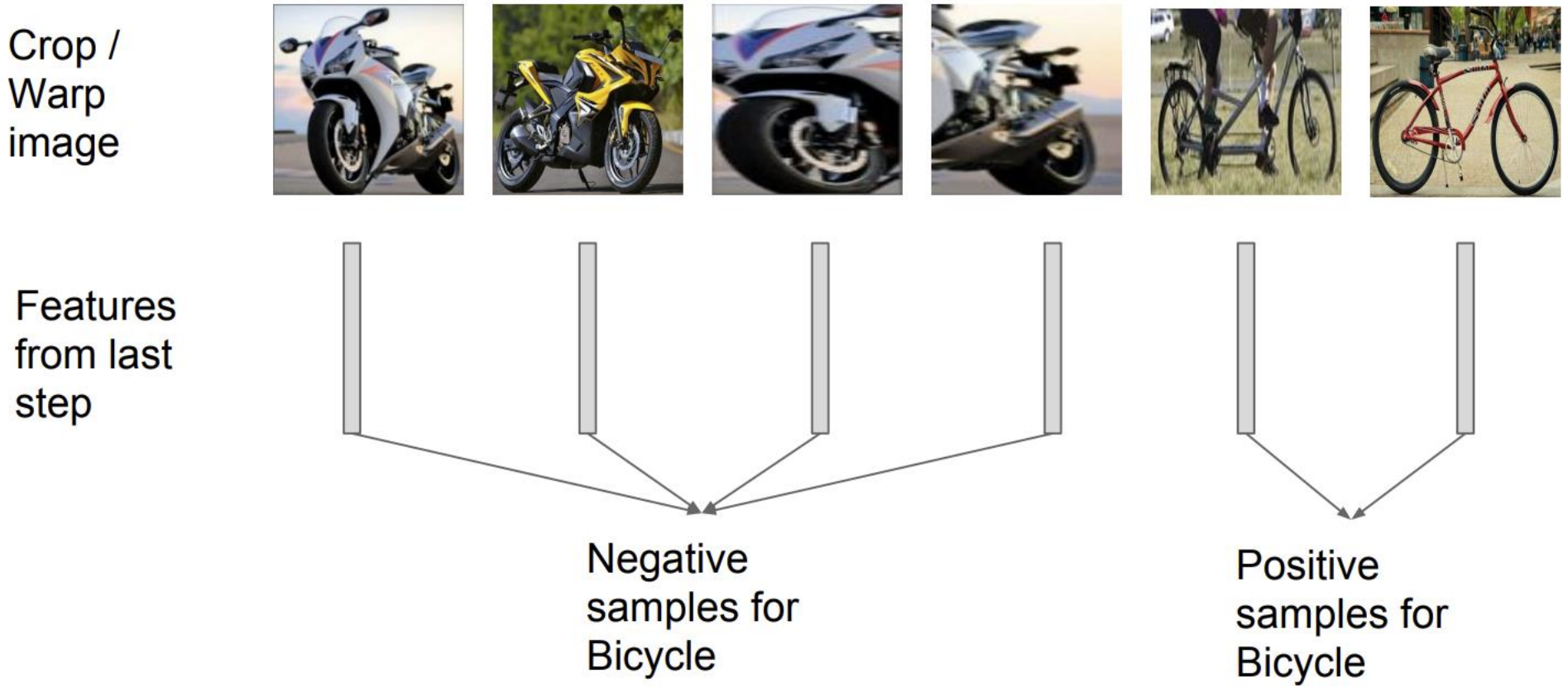
Training RCNN

- **Step 4**: train SVM for each class



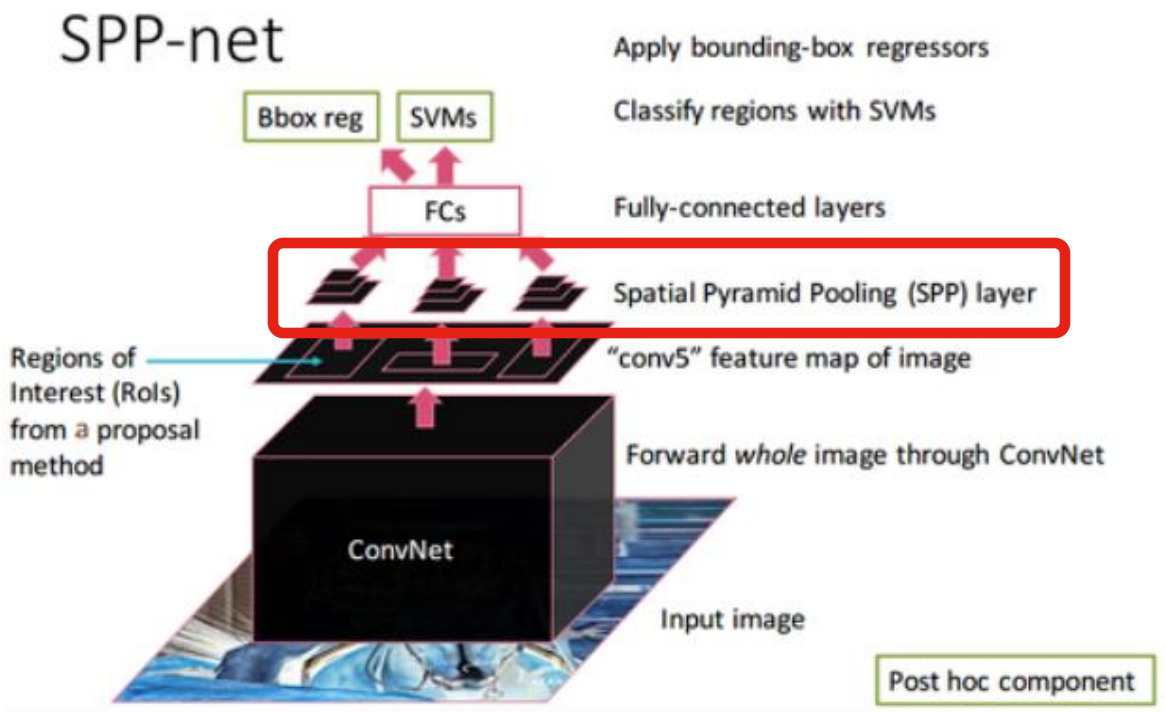
Training RCNN

- **Step 4**: train SVM for each class



Spatial Pyramid Pooling (SPP) Net

- Using single convolution to generate feature map

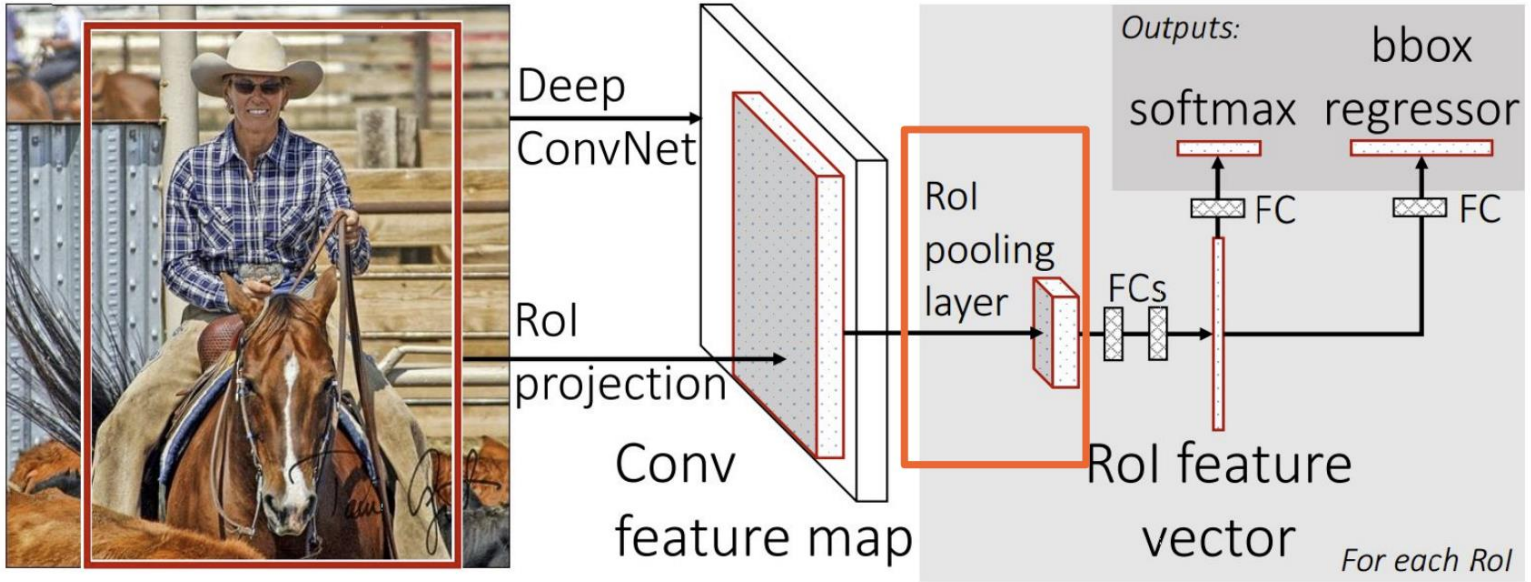


[Spatial Pyramid Pooling Network Architecture]

Image from: https://n1094.tistory.com/30

Fast R-CNN

- Share convolution layers for proposals from the same image
- Faster and More accurate than RCNN
- ROI Pooling



Ross Girshick, "**Fast R-CNN**", arxiv 2015

Fast R-CNN

- Single layer RoI pooling

Using 'Selective search'

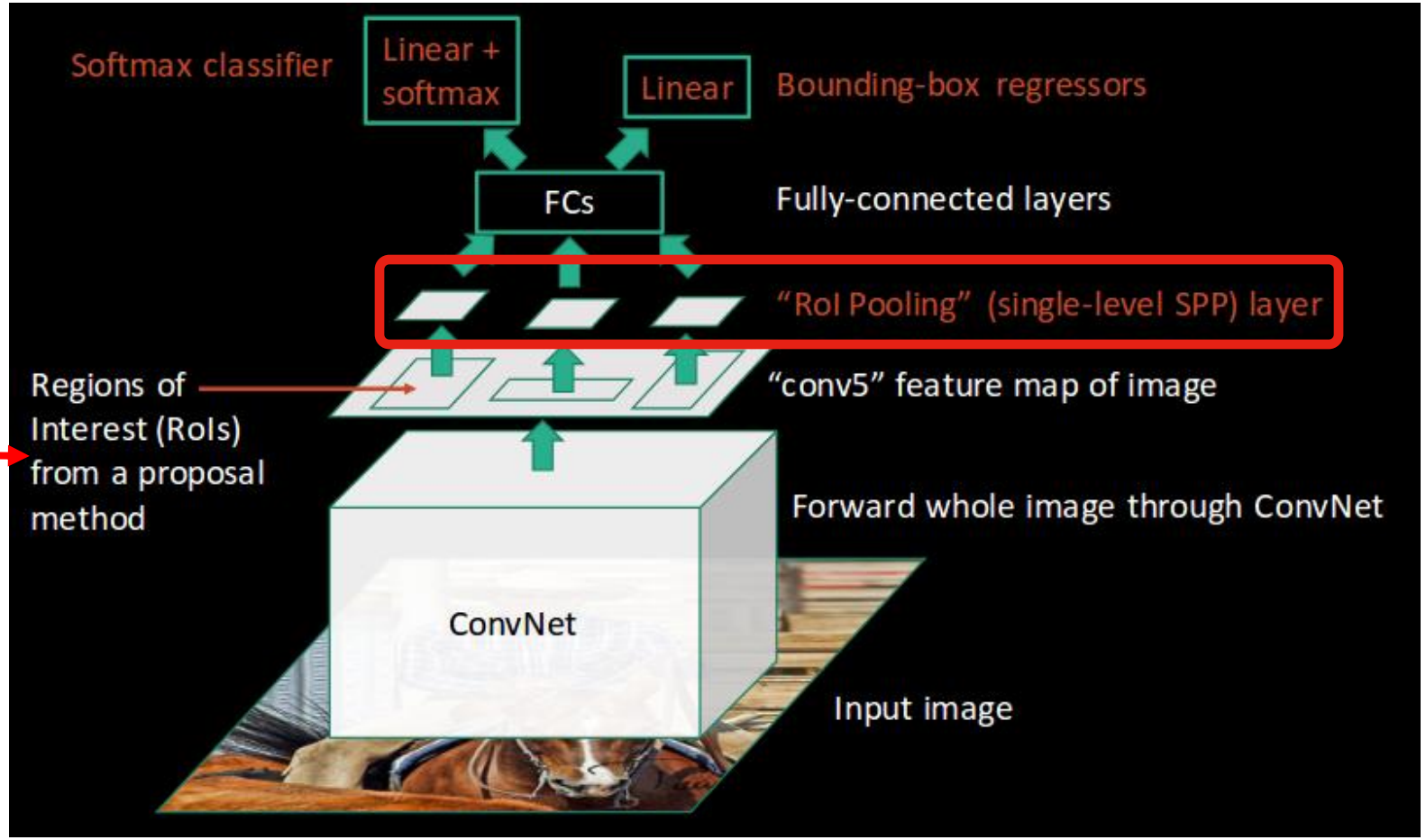
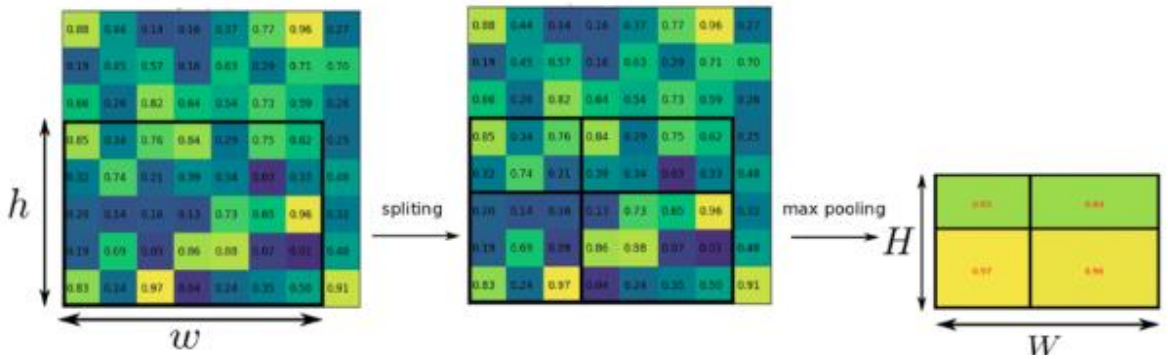
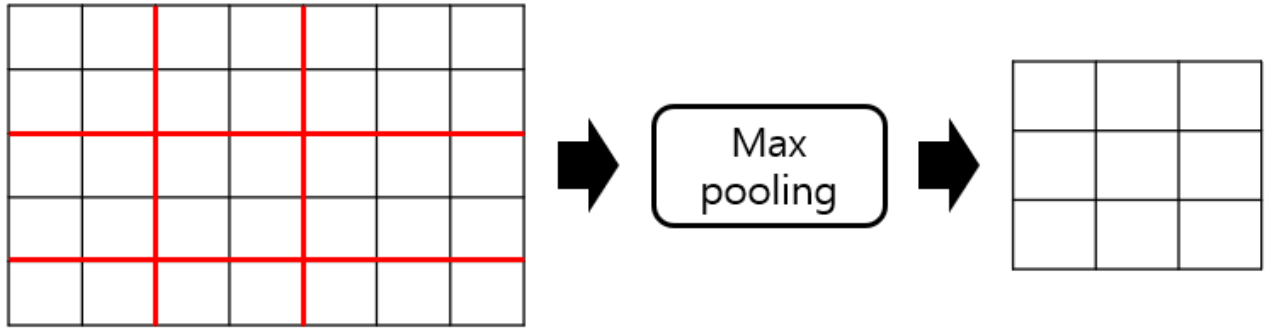


Image from: https://jaehyeongan.github.io/2019/10/10/R-CNN/

RoI Pooling

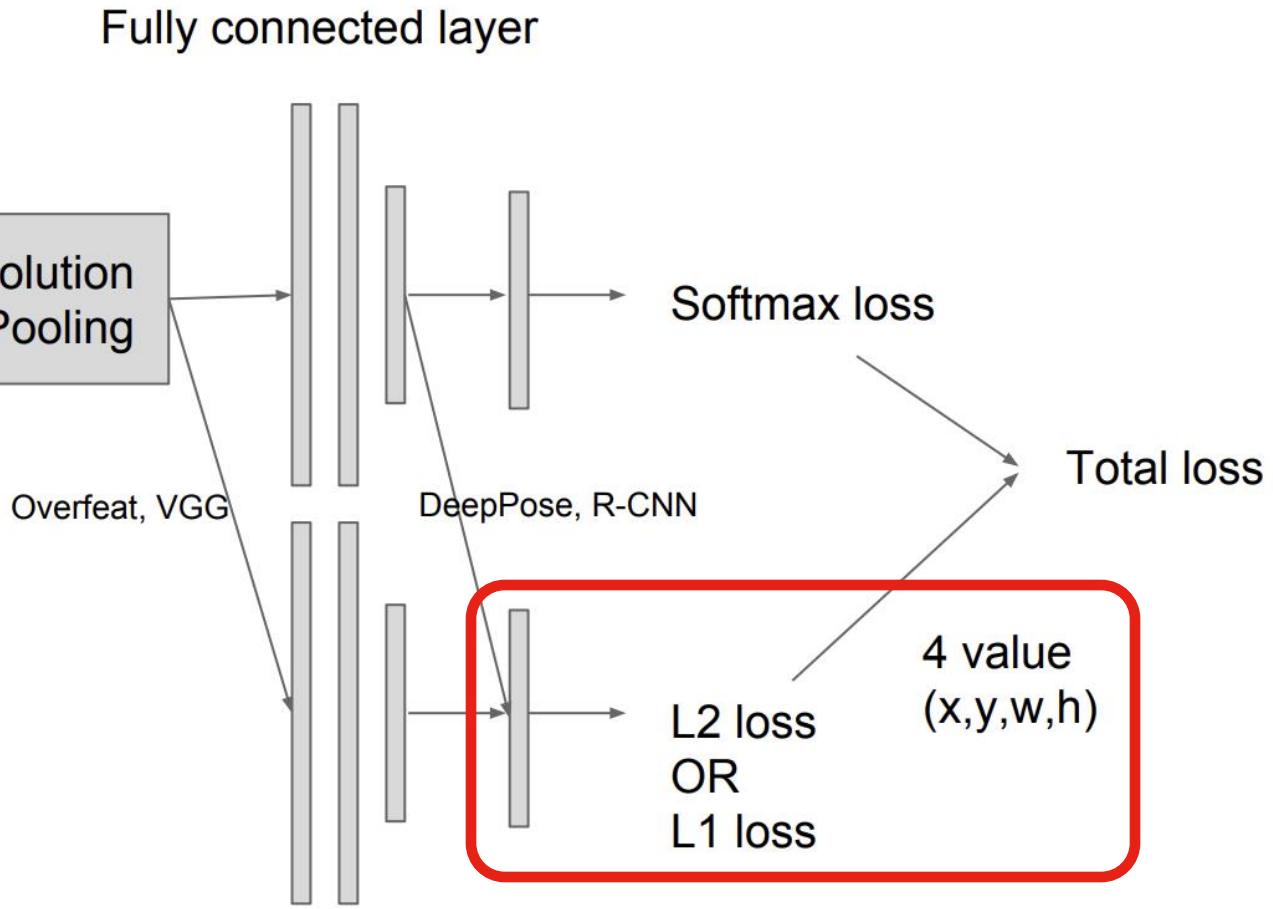
- To make the same sized output to classifier





What is bbox-regressor?

- Bounding box regression



Predicted box $P^i = (P^i_x, P^i_y, P^i_w, P^i_h)$

GND-truth box $G = (G_x, G_y, G_w, G_h)$.

Want to learn the P-moving function on feature

$$d_x(P),\ d_y(P),\ d_w(P),\ \text{and}\ d_h(P).$$

Result compare

- Trained using VGG 16 on Pascal VOC 2007 dataset
- Not including proposal time

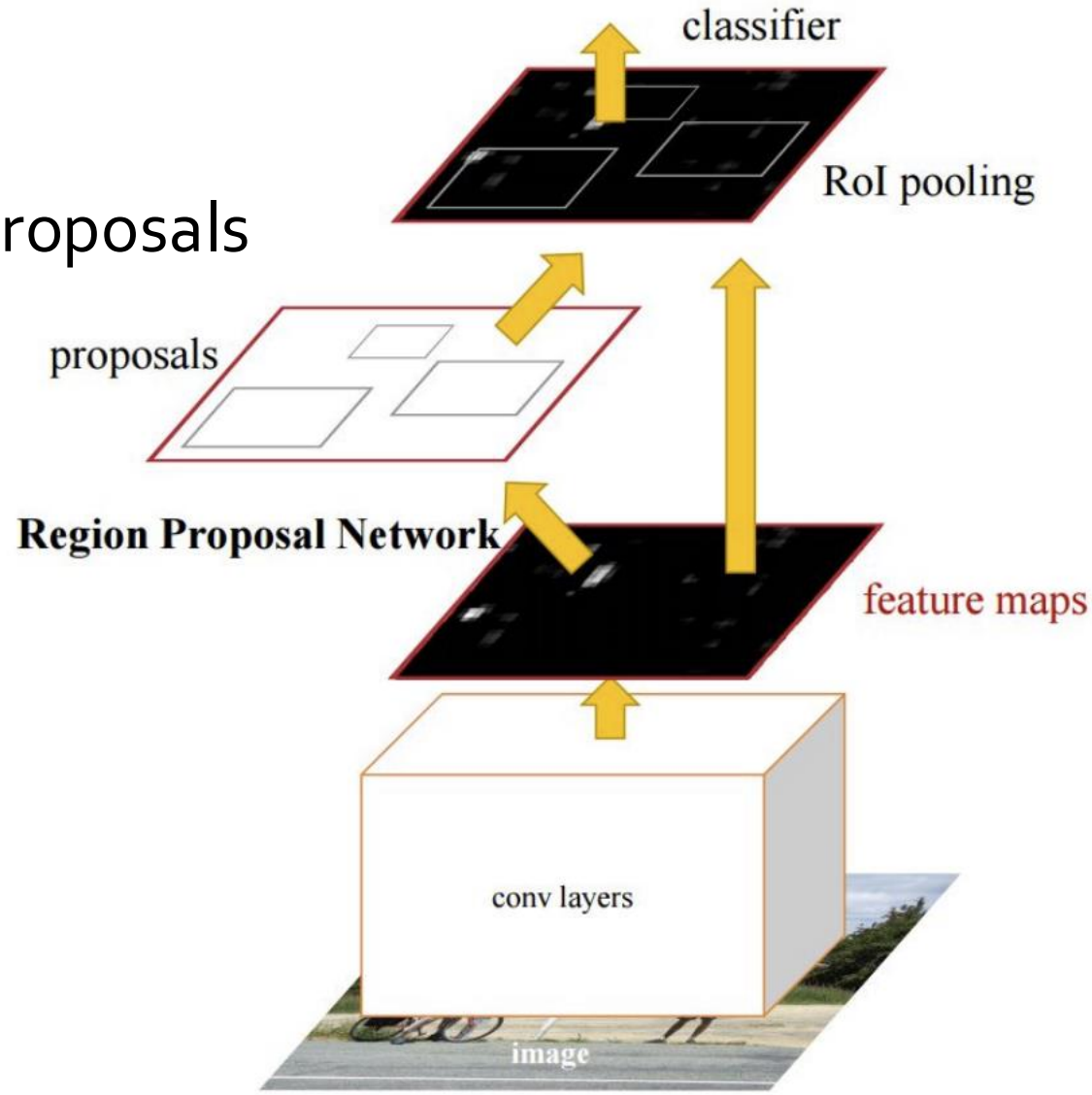
	Fast R-CNN	R-CNN
Train time (h)	9.5	84
-speedup	8.8x	1x
Test time/image	0.32s	47.00 s
-test speedup	146x	1x
mAP	66.9	66

Result compare (cont'd)

	Fast R-CNN	R-CNN
Test time/image	0.32s	47.00 s
-test speedup	146x	1x
Test time/image with proposal	2s	50 s
-test speedup	25x	1x

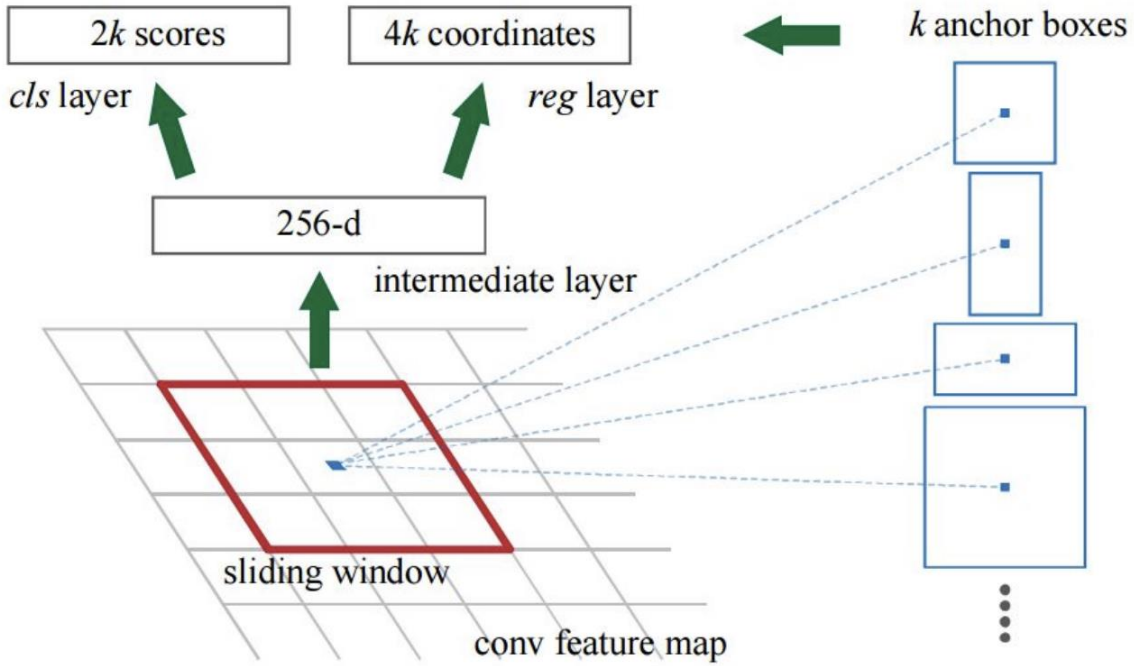
Faster RCNN

- Don’t need to have external regional proposals
- RPN - Regional Proposal Network



Ren, He, Girshick, Sun, “**Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks**” *ICCV 2015*

Region Proposal Network (RPN)



Result compare

- Trained using Pascal VOC 2007 dataset

	Faster R-CNN	Fast R-CNN	R-CNN
Test time/image With proposal	0.2S	2s	50s
-test speedup	250x	25x	1x
mAP	66.9	66.9	66

R-FCN :Region-based Fully Convolutional Networks

- Use position sensitive score map
- Share all conv and fc layers between all proposals for the same image

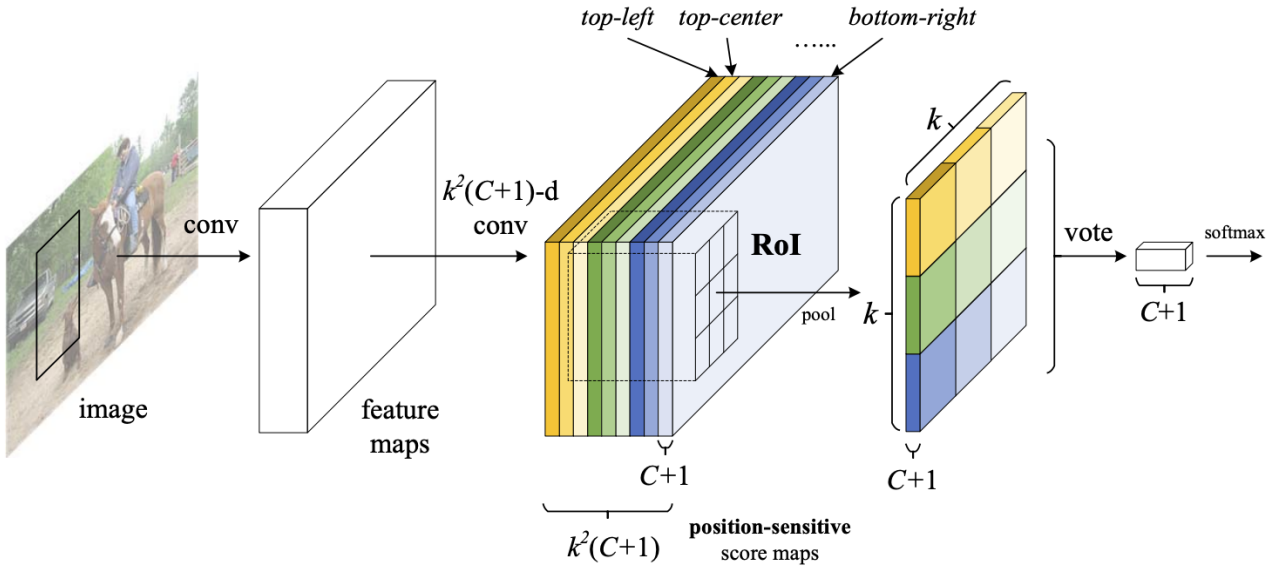


Figure 1: Key idea of **R-FCN** for object detection. In this illustration, there are $k \times k = 3 \times 3$ position-sensitive score maps generated by a fully convolutional network. For each of the $k \times k$ bins in an RoI, pooling is only performed on one of the k^2 maps (marked by different colors).

Dai, Li, He, Sun, "**R-FCN: Object Detection via Region-based Fully Convolutional Networks**", *NeurIPS 2016*

R-FCN

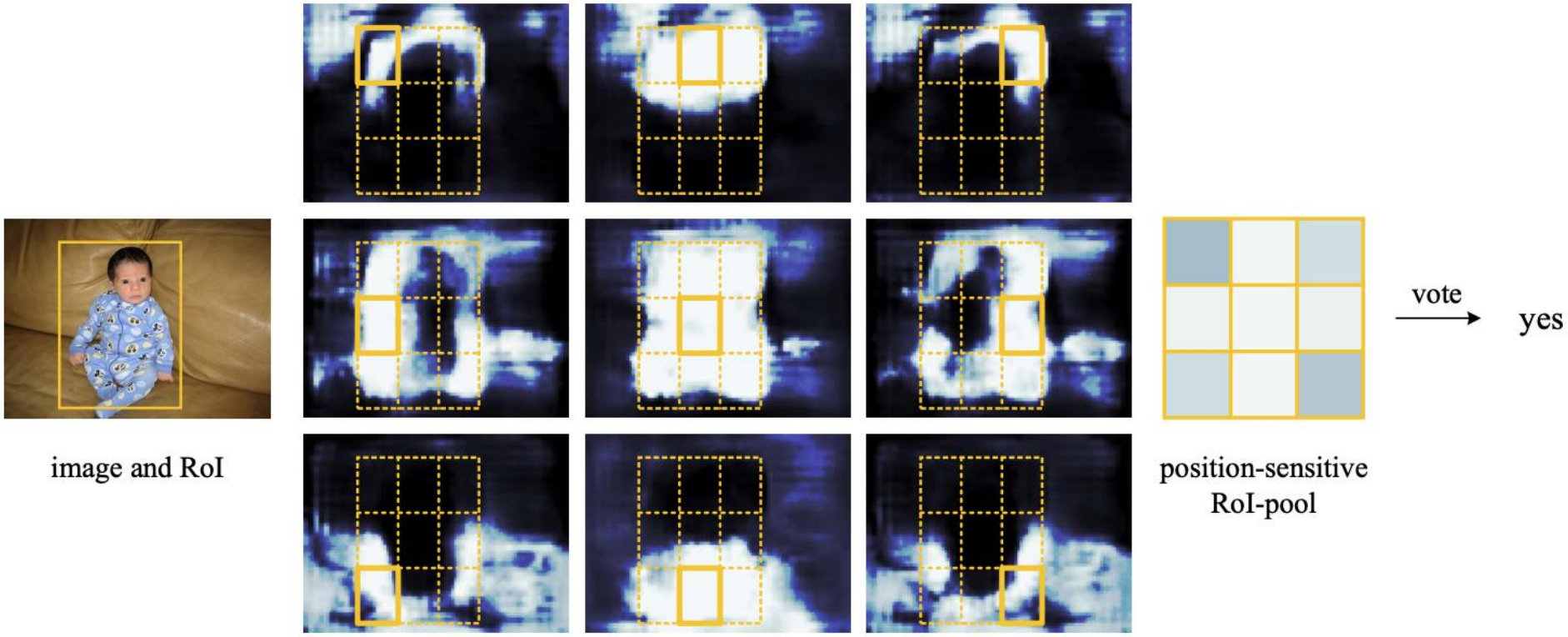


Figure 3: Visualization of R-FCN ($k \times k = 3 \times 3$) for the *person* category.

R-FCN

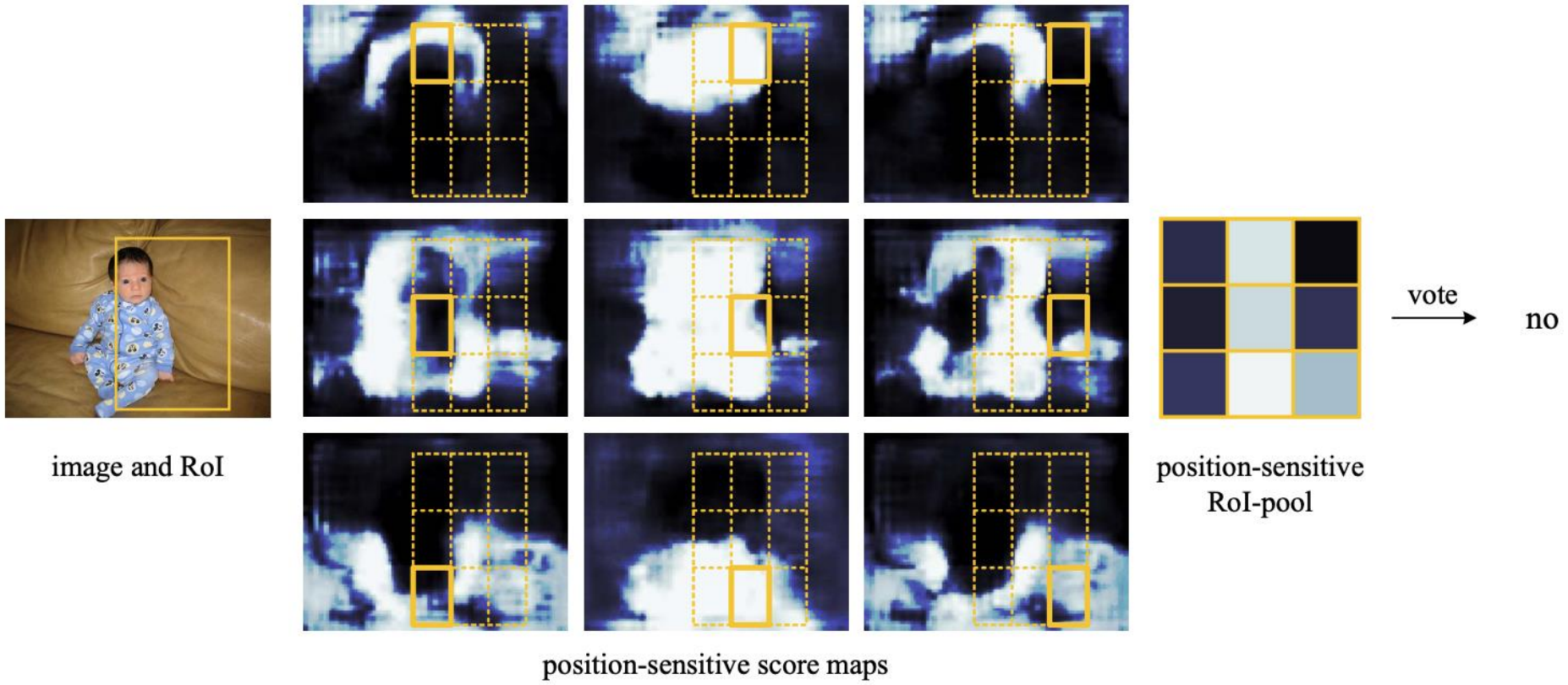


Figure 4: Visualization when an RoI does not correctly overlap the object.

Performance comparison

	RCNN	Faster RCNN	RFCN
Depth of shared convolutional subnetwork	0	91	101
Depth of ROI-wise subnetwork	101	10	0

Trained using ResNet 101

	Faster R-CNN	R-FCN
Test time/image With proposal	0.42S	0.2s
mAP	76.4	76.6

Trained using ResNet 101 on Pascal VOC 2007 dataset

Fast RCNN vs Faster RCNN

	method	# proposals	data	mAP (%)	time (ms)
Fast RCNN	SS	2k	07	66.9†	1830
	SS	2k	07+12	70.0	1830
Faster RCNN	RPN+VGG, unshared	300	07	68.5	342
	RPN+VGG, shared	300	07	69.9	**198**
	RPN+VGG, shared	300	07+12	**73.2**	**198**

Using CNN in two ways

- Two stage detection
  - Slow
  - Accurate
  - E.g., R-CNN and its variants (Fast R-CNN, Faster R-CNN)

- Single stage detection
  - Fast
  - Less accurate (especially for the small objects)
  - E.g., YOLO, SSD

Problems with two-stage detection methods

- Complex Pipeline
- Slow (hard to run in real time)

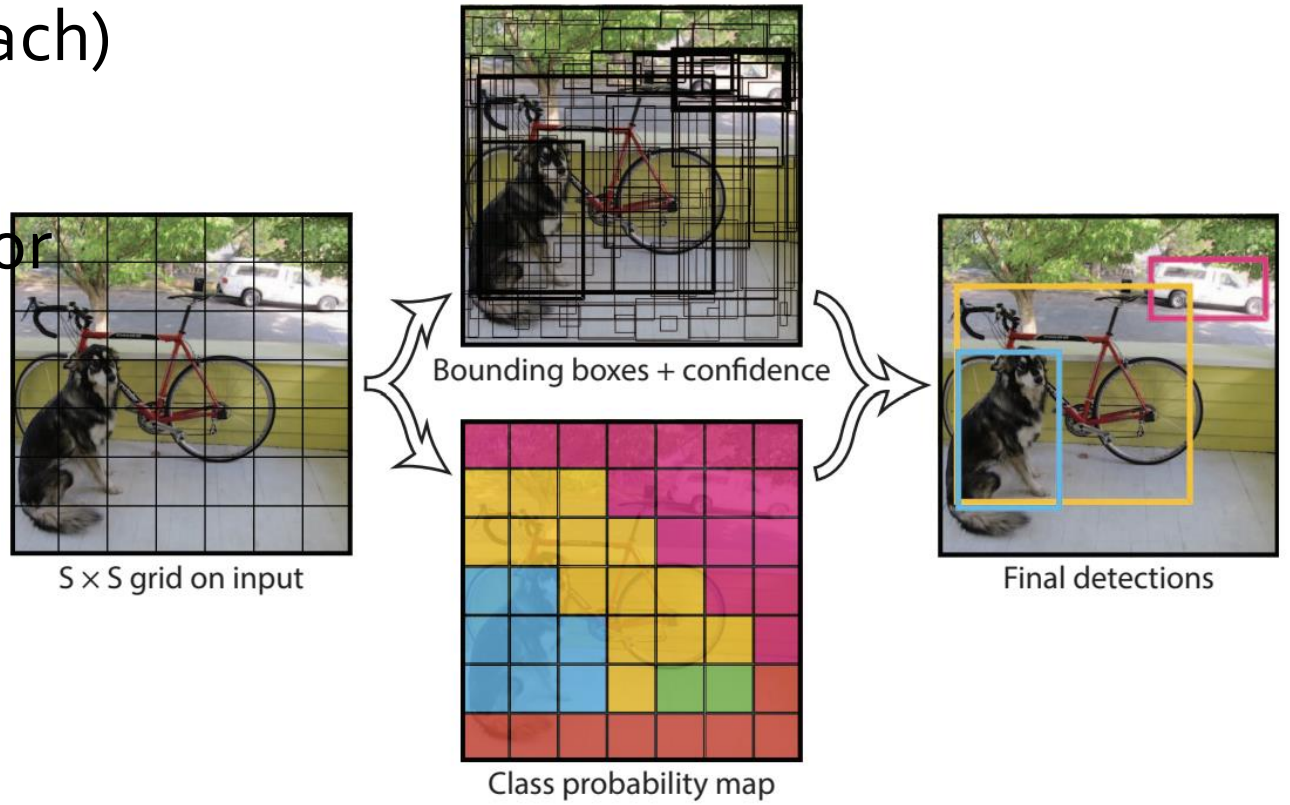
YOLO: You Only Look Once

- Consider detection a regression problem
- Use a single ConvNet
- Runs once on entire image
  - Convolution is inherently sliding window!
- Very Fast!
- Great naming

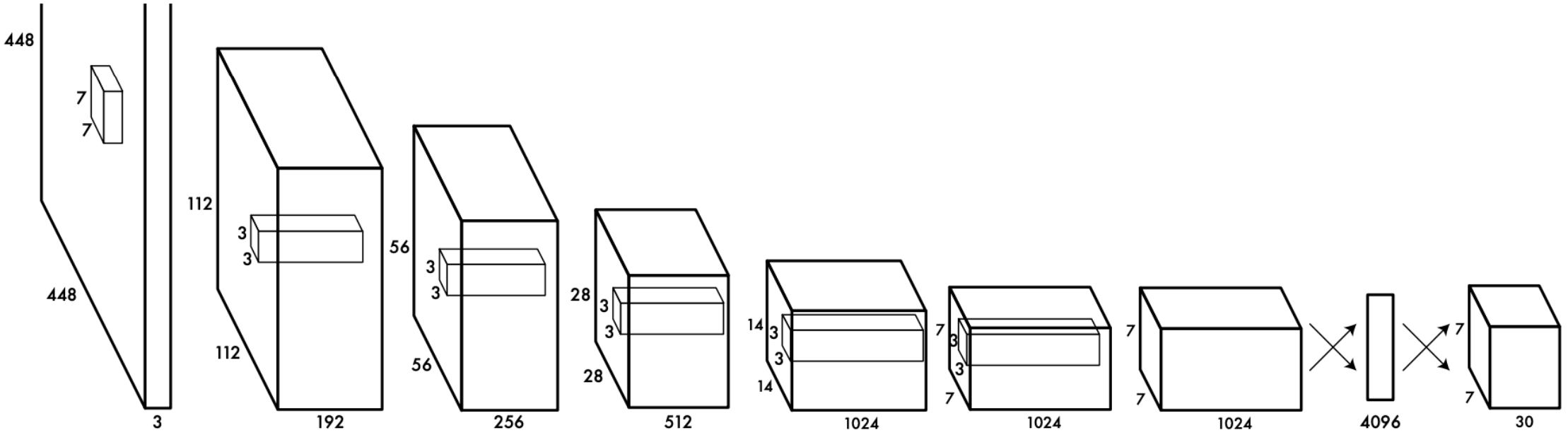
Redmon, Divvala, Girshick, Farhadi, "**You only look once: Unified, real-time object detection**", CVPR 2016

How YOLO works

- The following predictions are made for each cell in an $S \times S$ grid
- C conditional class probabilities $\Pr(Class \mid Obj)$
- B bounding boxes (4 parameters each)
  - B confidence scores $\Pr(Obj) \times IoU$
- Output is $S \times S \times (5B + C)$ tensor



Network architecture



Performance (VOC 2007)

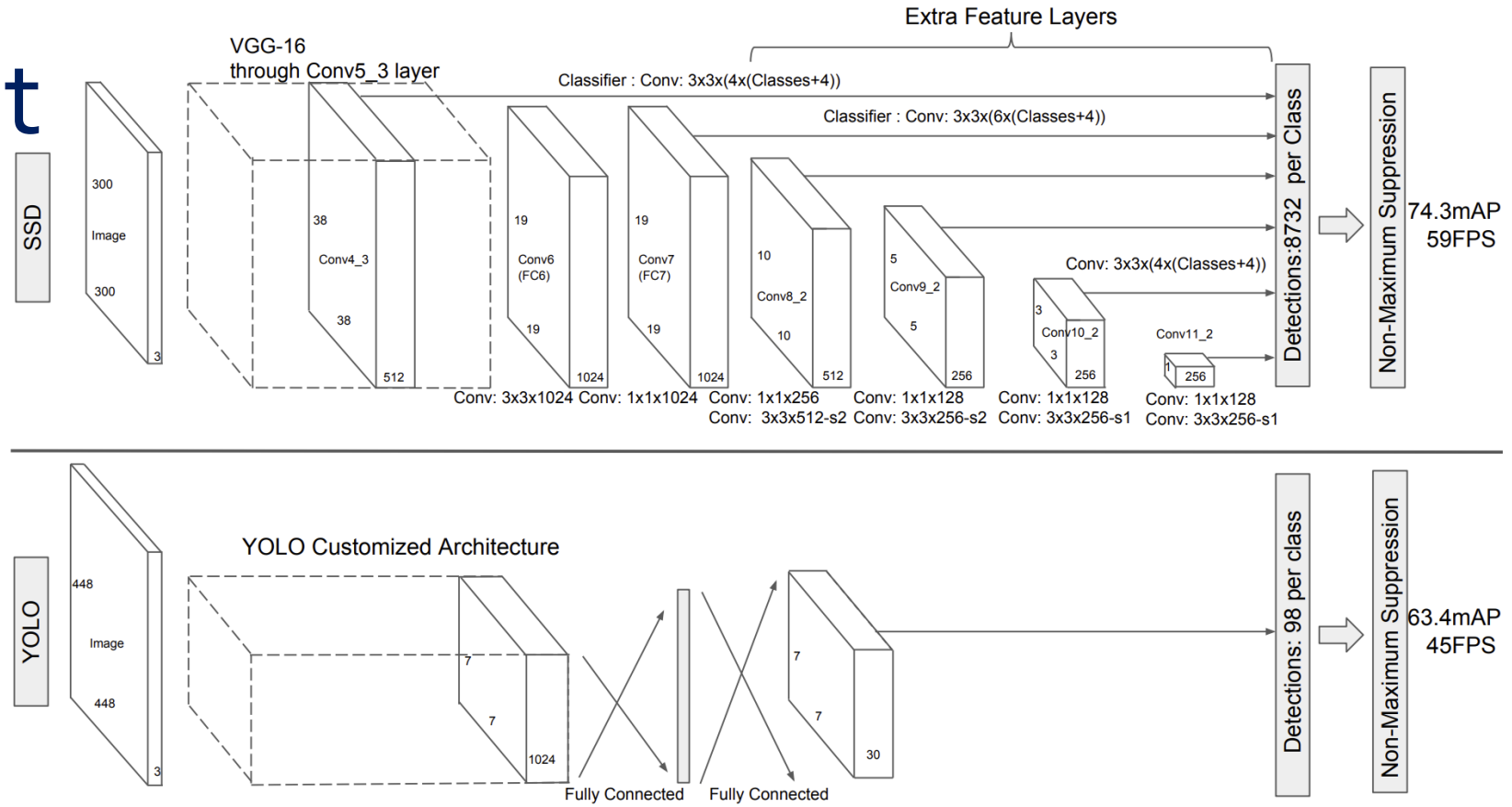
- Trained on Pascal VOC 2007 + 2012 dataset

	Yolo	Faster R-CNN (VGG-16)
mAP	63.4	73.2
FPS	45	7

Limitations

- Struggles with small objects
- Struggles with unusual aspect ratios
- Poor localization

SSD: Single Shot Detector

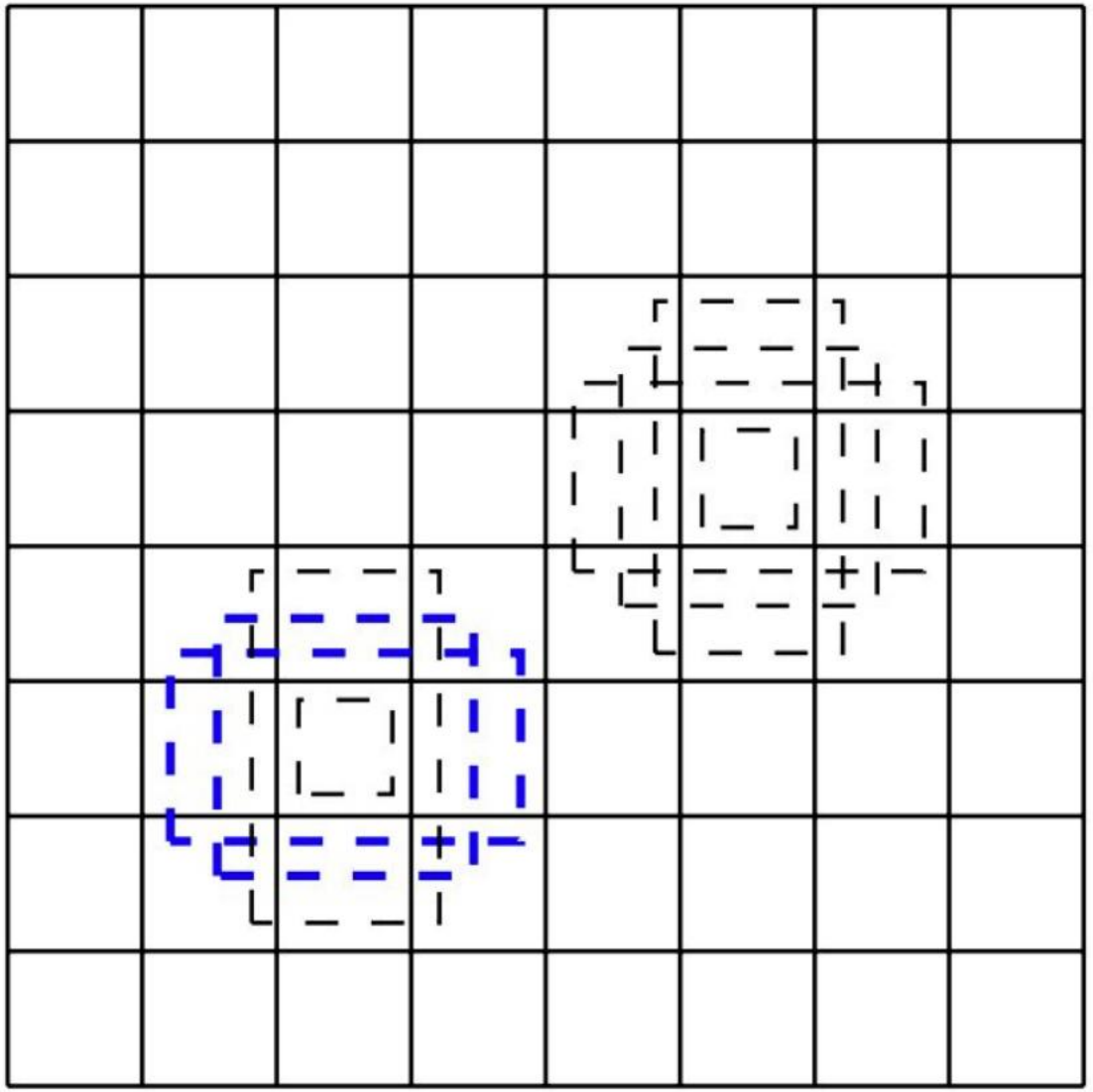


- Faster than YOLO, as accurate as Faster R-CNN
- Uses small convolutional filters applied to feature maps
- **Makes predictions using feature maps of different scales**

Liu et. al., "**SSD: Single shot multibox detector**" *ECCV 2016*

Default Boxes

- Multiple aspect ratios per cell
- Similar to Faster R-CNN's Anchor Boxes
  - Applied to many feature maps.



SSD Detector

- Detectors are convolutional filters
- Each detector outputs a single value
- Predict **class probabilities** and **bounding box offsets**
- (classes + 4) detectors are needed for a detection
- (classes + 4) x (#default boxes) x m x n outputs for a m x n feature map

Results (VOC 2007)

	SSD*	Yolo*	Faster R-CNN*
mAP	74.3	66.4	73.2
FPS	46	21	7

- Trained on Pascal VOC 2007 + 2012 dataset
- *VGG16

YOLO V2: Faster, Better Stronger

	YOLO							YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓
hi-res classifier?			✓	✓	✓	✓	✓	✓
convolutional?				✓	✓	✓	✓	✓
anchor boxes?				✓	✓			
new network?					✓	✓	✓	✓
dimension priors?						✓	✓	✓
location prediction?						✓	✓	✓
passthrough?							✓	✓
multi-scale?								✓
hi-res detector?								✓
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8	**78.6**

Redmon, Farhadi, "**YOLO9000: Better, Faster, Stronger**", *CVPR 2017* (best paper honorable mention)

Performance on MS COCO

		0.5:0.95	0.5	0.75	S	M	L	1	10	100	S	M	L
Fast R-CNN [5]	train	19.7	35.9	-	-	-	-	-	-	-	-	-	-
Fast R-CNN[1]	train	20.5	39.9	19.4	4.1	20.0	35.8	21.3	29.5	30.1	7.3	32.1	52.0
Faster R-CNN[15]	trainval	21.9	42.7	-	-	-	-	-	-	-	-	-	-
ION [1]	train	23.6	43.2	23.6	6.4	24.1	38.3	23.2	32.7	33.5	10.1	37.7	53.6
Faster R-CNN[10]	trainval	24.2	45.3	23.5	7.7	26.4	37.1	23.8	34.0	34.6	12.0	38.5	54.4
SSD300 [11]	trainval35k	23.2	41.2	23.4	5.3	23.2	39.6	22.5	33.2	35.3	9.6	37.6	56.5
SSD512 [11]	trainval35k	**26.8**	**46.5**	**27.8**	**9.0**	**28.9**	**41.9**	**24.8**	**37.5**	**39.8**	**14.0**	**43.5**	**59.0**
YOLOv2 [11]	trainval35k	21.6	44.0	19.2	5.0	22.4	35.5	20.7	31.6	33.3	9.8	36.5	54.4

Characteristic of deep CNN for detection

- As CNN gets deeper
  - Better abstraction of information
  - **Good**: Better for classification
  - **Bad**: Spatial resolution gets smaller → spatial information loss → bad localization performance

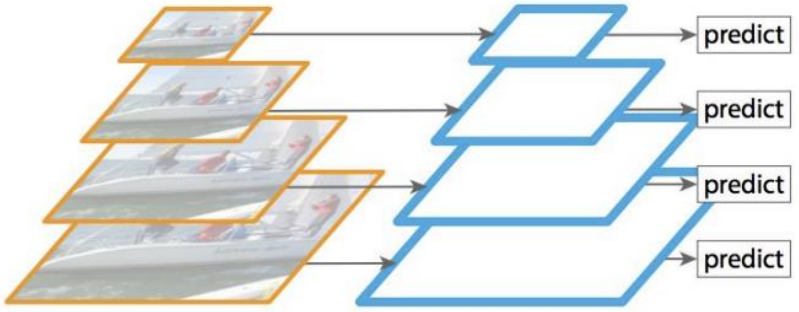
Feature Pyramid Networks (FPN)

- Uses ConvNet as Feature Pyramid
- Includes **low level feature maps to detect small objects**
  - Particularly problematic in single stage detector
- Top down pathway provides contextual information
- Makes single stage detector **fast and accurate**

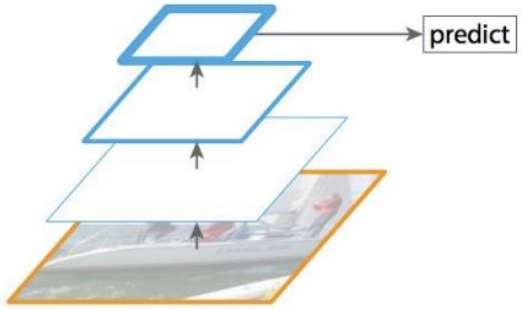
Lin, Dollár, Girshick, He, Hariharan, Belongie, "**Feature Pyramid Networks for Object Detection**", *CVPR 2017*

FPN compared to prior methods

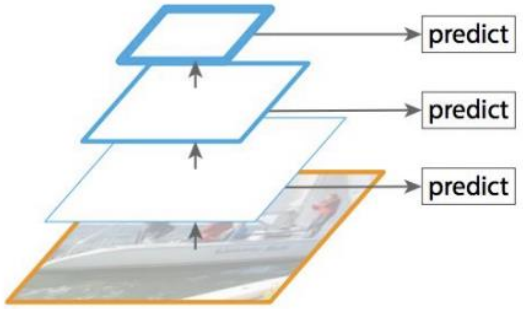
- Accurate but slow
- Misses low level information (YOLO)
- Misses context in low level predictions (SSD)
- Accurate and fast



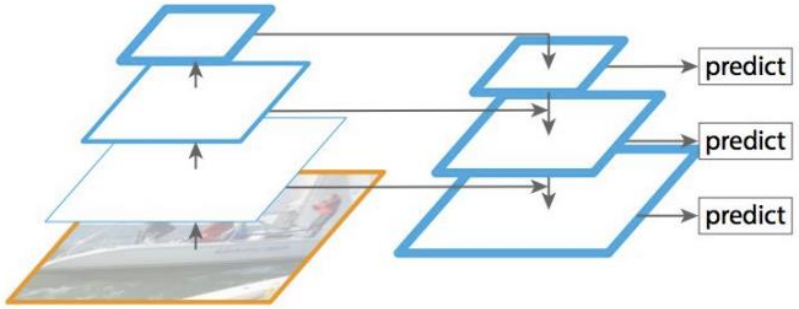
(a) Featurized image pyramid



(b) Single feature map



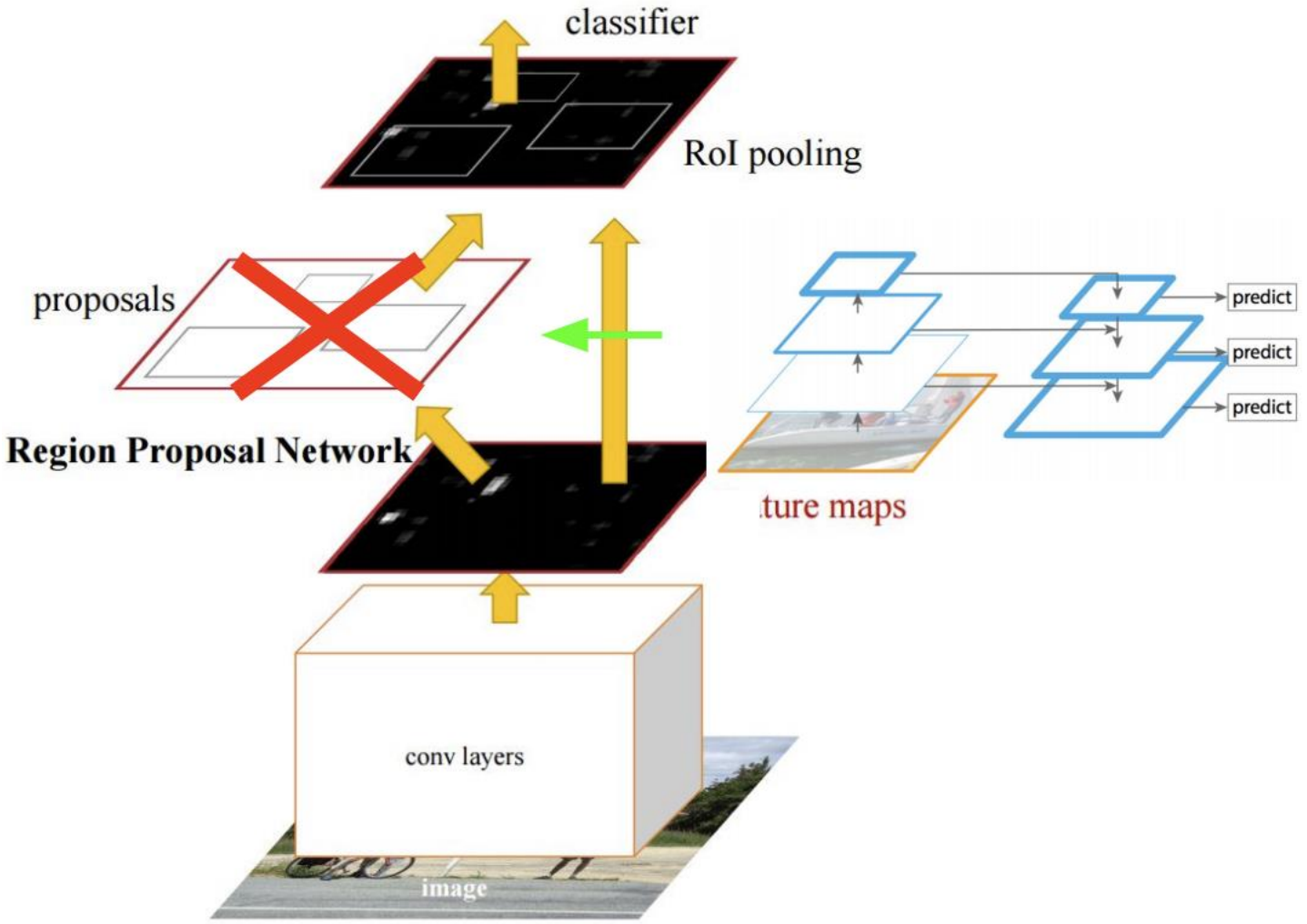
(c) Pyramidal feature hierarchy



(d) Feature Pyramid Network

Feature Pyramid Networks for RPN

- Replace single scale feature map with FPN
- Single scale anchors at each level
- **Also applicable to single stage detectors!**



Results

- COCO
- State of the art single-model

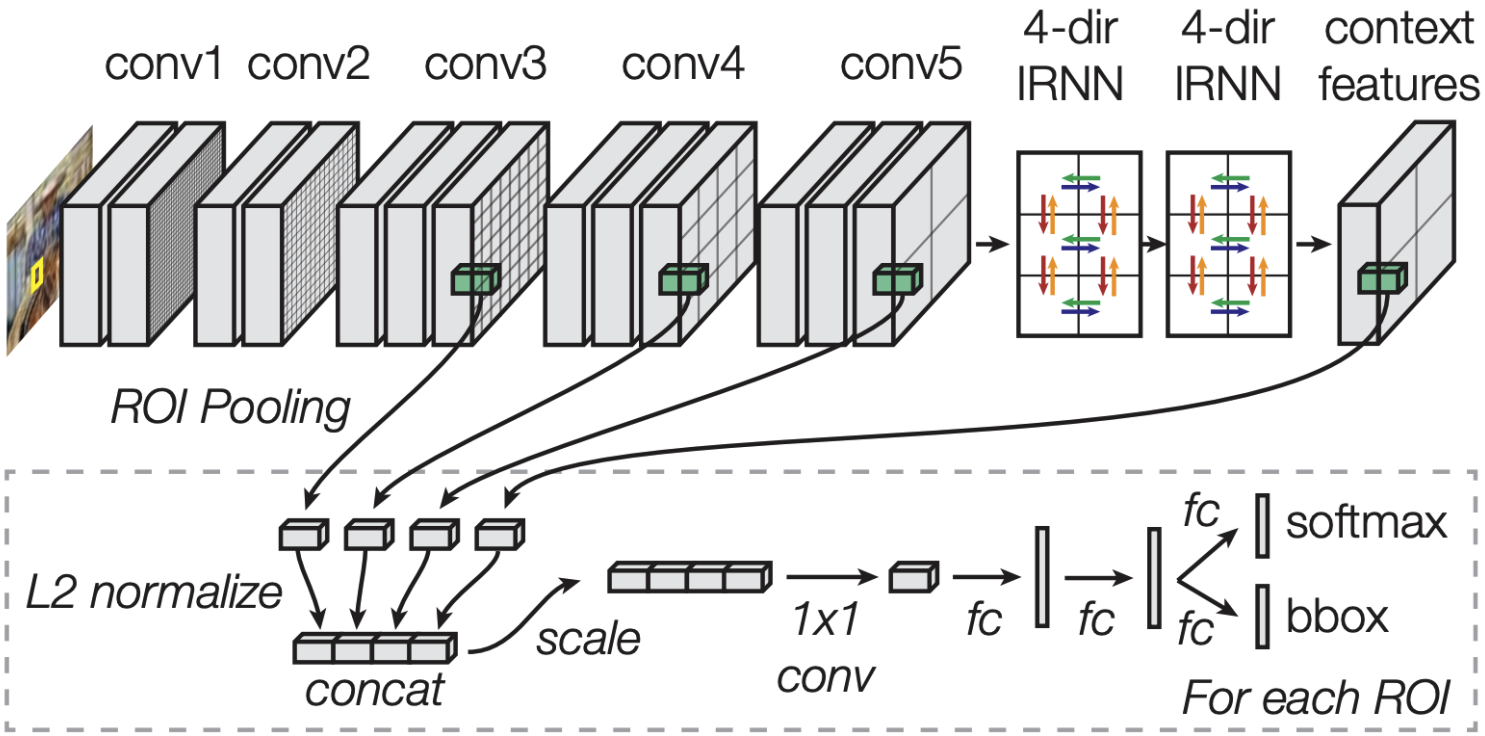
	Faster R-CNN on **FPN***	Faster R-CNN +++*	ION**
mAP	36.2	34.9	31.2
mAP (small images)	18.2	15.6	12.8

* ResNet-101

** VGG-16

ION : Inside-Outside Network

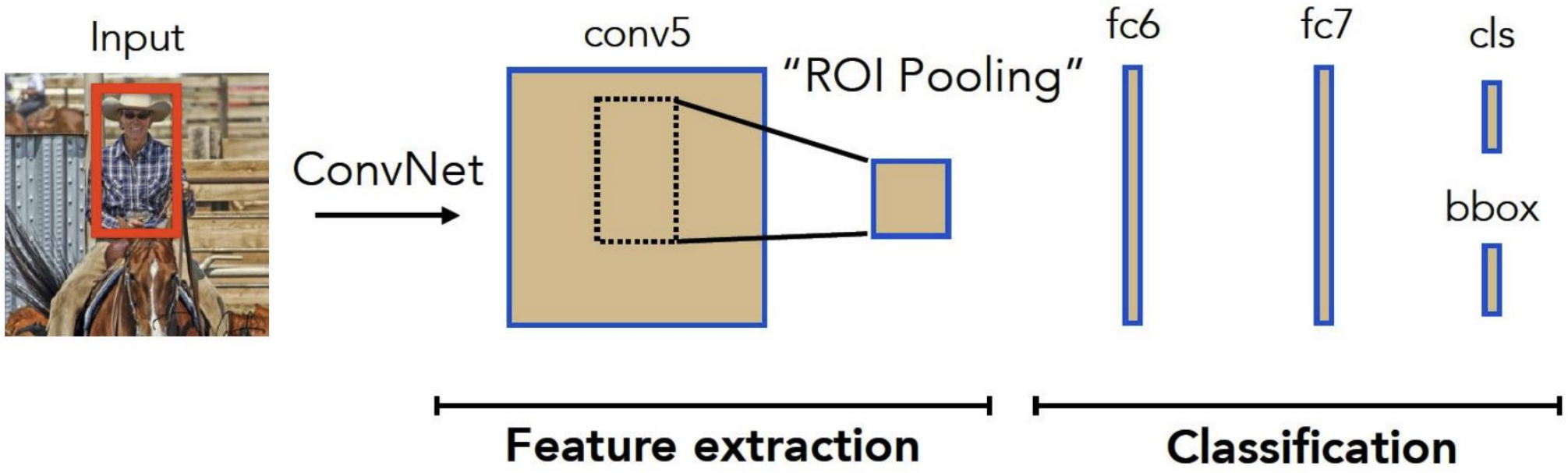
- Use **multi-scale Conv features** for inside region
- Use four direction RNN features for outside region



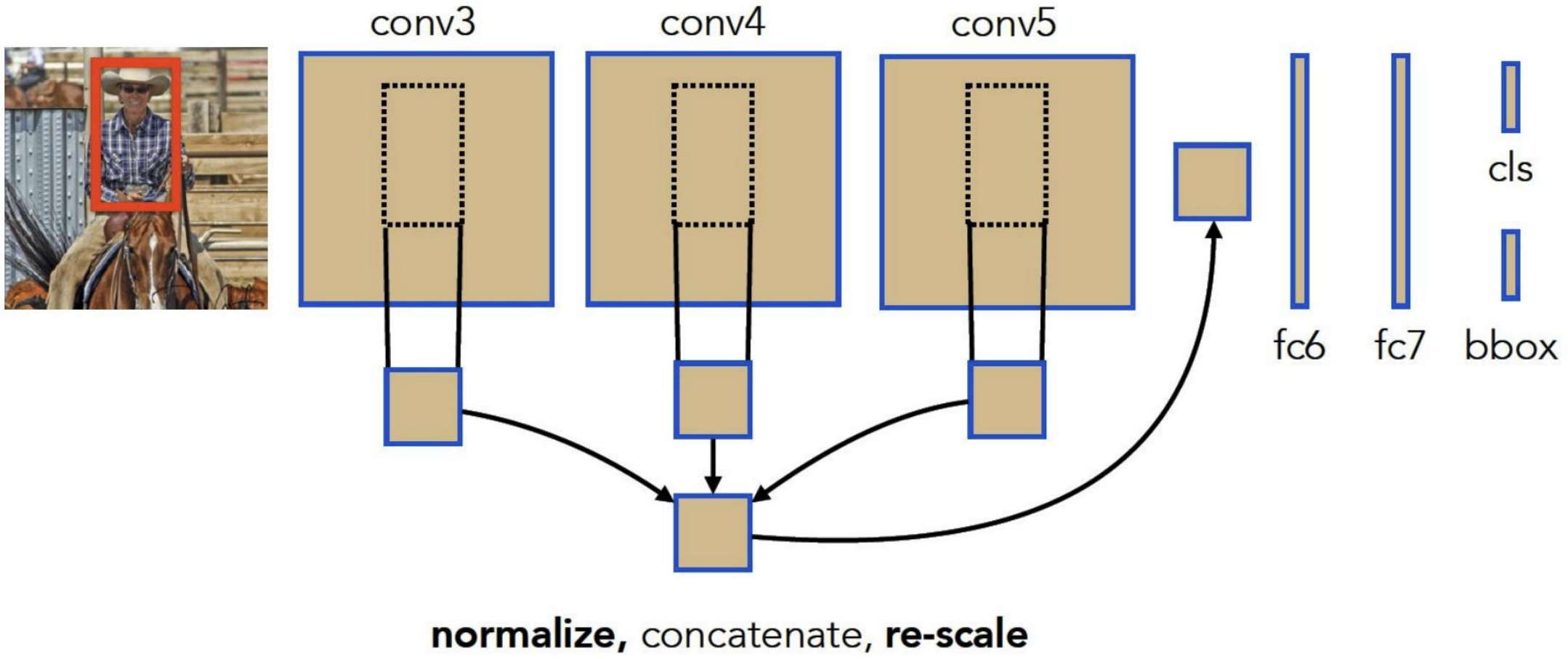
Bell et al., "**Inside-Outside Net: Detecting Objects in Context with Skip Pooling and Recurrent Neural Networks**", CVPR 2016

Can we improve on feature extraction?

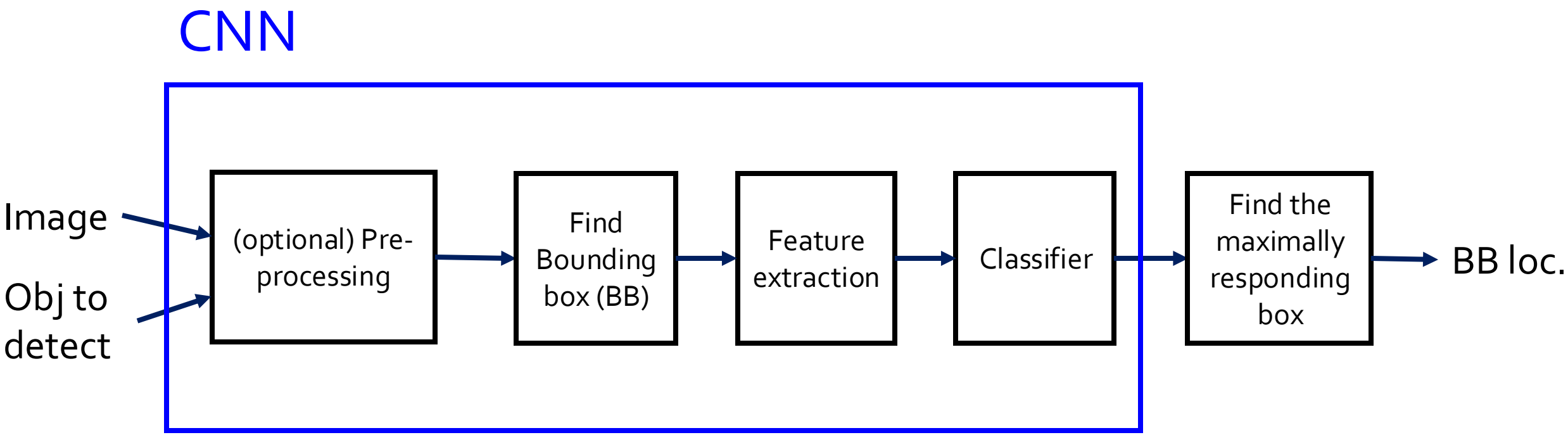
- For small objects, the footprint on conv5 might only cover a 1x1 cell, which gets upsampled to 7x7
- Only local features (inside the ROI) are used for classification



Combining across layers



New pipeline



Find the maximally responding box

- A super popular heuristic: **Non Maximum Suppression (NMS)**

Find the maximally responding box

- A super popular heuristic: **Non Maximum Suppression (NMS)**

Algorithm 1 Non-Max Suppression

1: **procedure** NMS(B,c)
2: $B_{nms} \leftarrow \emptyset$ Initialize empty set
3: **for** $b_i \in B$ **do** => Iterate over all the boxes
4: $discard \leftarrow$ False Take boolean variable and set it as false. This variable indicates whether b(i) should be kept or discarded
5: **for** $b_j \in B$ **do** Start another loop to compare with b(i)
6: **if** same$(b_i, b_j) > \boldsymbol{\lambda_{nms}}$ **then** If both boxes having same IOU
7: **if** score$(c, b_j) >$ score(c, b_i) **then**
8: $discard \leftarrow$ True Compare the scores. If score of b(i) is less than that of b(j), b(i) should be discarded, so set the flag to True.
9: **if not** $discard$ **then** Once b(i) is compared with all other boxes and still the discarded flag is False, then b(i) should be considered. So add it to the final list.
10: $B_{nms} \leftarrow B_{nms} \cup b_i$
11: **return** B_{nms} Do the same procedure for remaining boxes and return the final list

Can we do better than NMS?

- NMS

$$s_i = \begin{cases} s_i, & \text{iou}(\mathcal{M}, b_i) < N_t \\ 0, & \text{iou}(\mathcal{M}, b_i) \geq N_t \end{cases},$$

- Proposed Soft-NMS

$$s_i = \begin{cases} s_i, & \text{iou}(\mathcal{M}, b_i) < N_t \\ s_i(1 - \text{iou}(\mathcal{M}, b_i)), & \text{iou}(\mathcal{M}, b_i) \geq N_t \end{cases},$$

$$s_i = s_i e^{-\frac{\text{iou}(\mathcal{M}, b_i)^2}{\sigma}}, \forall b_i \notin \mathcal{D}$$

Input : $\mathcal{B} = \{b_1, .., b_N\}, \mathcal{S} = \{s_1, .., s_N\}, N_t$
$\mathcal{B}$ is the list of initial detection boxes
$\mathcal{S}$ contains corresponding detection scores
N_t is the NMS threshold

begin
  $\mathcal{D} \leftarrow \{\}$
  while $\mathcal{B} \neq$ *empty* **do**
    $m \leftarrow$ argmax $\mathcal{S}$
    $\mathcal{M} \leftarrow b_m$
    $\mathcal{D} \leftarrow \mathcal{D} \bigcup \mathcal{M}; \mathcal{B} \leftarrow \mathcal{B} - \mathcal{M}$
    for b_i *in* $\mathcal{B}$ **do**
      if $iou(\mathcal{M}, b_i) \geq N_t$ **then**
        $\mathcal{B} \leftarrow \mathcal{B} - b_i; \mathcal{S} \leftarrow \mathcal{S} - s_i$
      end NMS
      $s_i \leftarrow s_i f(iou(\mathcal{M}, b_i))$ Soft-NMS
    end
  end
  return $\mathcal{D}, \mathcal{S}$
end

Bodla, Singh, Chellappa, Davis, "**Soft-NMS - Improving Object Detection With One Line of Code**", *ICCV 2017*

Performance of Soft-NMS

Method	Training data	Testing data	AP 0.5:0.95	AP @ 0.5	AP small	AP medium	AP large	Recall @ 10	Recall @ 100
R-FCN [16]	train+val35k	test-dev	31.1	52.5	14.4	34.9	43.0	42.1	43.6
R-FCN + S-NMS G	train+val35k	test-dev	**32.4**	**53.4**	**15.2**	**36.1**	**44.3**	**46.9**	**52.0**
R-FCN + S-NMS L	train+val35k	test-dev	32.2	**53.4**	15.1	36.0	44.1	46.0	51.0
F-RCNN [24]	train+val35k	test-dev	24.4	45.7	7.9	26.6	37.2	36.1	37.1
F-RCNN + S-NMS G	train+val35k	test-dev	**25.5**	46.6	**8.8**	**27.9**	**38.5**	**41.2**	45.3
F-RCNN + S-NMS L	train+val35k	test-dev	**25.5**	**46.7**	**8.8**	**27.9**	38.3	40.9	**45.5**
D-RFCN [3]	trainval	test-dev	37.4	59.6	17.8	40.6	51.4	46.9	48.3
D-RFCN S-NMS G	trainval	test-dev	**38.4**	**60.1**	**18.5**	**41.6**	**52.5**	**50.5**	**53.8**
D-RFCN + MST	trainval	test-dev	39.8	62.4	22.6	42.3	52.2	50.5	52.9
D-RFCN + MST + S-NMS G	trainval	test-dev	**40.9**	**62.8**	**23.3**	**43.6**	**53.3**	**54.7**	**60.4**

Table 1. Results on MS-COCO test-dev set for R-FCN, D-RFCN and Faster-RCNN (F-RCNN) which use NMS as baseline and our proposed Soft-NMS method. G denotes Gaussian weighting and L denotes linear weighting. MST denotes multi-scale testing.

Method	AP	AP@0.5	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
[24]+NMS	37.7	70.0	37.8	44.6	34.7	24.4	23.4	50.6	50.1	45.1	25.1	42.6	36.5	40.7	46.8	39.8	38.2	17.0	37.8	36.4	43.7	**38.9**
[24]+S-NMS G	**39.4**	**71.2**	40.2	**46.6**	**36.7**	25.9	**24.9**	**51.9**	51.6	**48.0**	**25.3**	44.5	**37.3**	42.6	**49.0**	42.2	41.6	17.7	39.2	**39.3**	**45.9**	37.5
[24]+S-NMS L	**39.4**	**71.2**	**40.3**	**46.6**	36.3	**27.0**	24.2	51.2	**52.0**	47.2	**25.3**	**44.6**	37.2	**45.1**	48.3	**42.3**	**42.3**	**18.0**	**39.4**	37.1	45.0	38.7
[16]+NMS	49.8	79.4	52.8	54.4	47.1	37.6	38.1	63.4	59.4	62.0	35.3	56.0	38.9	59.0	54.5	50.5	47.6	24.8	53.3	52.2	57.4	52.7
[16]+S-NMS G	51.4	**80.0**	**53.8**	**56.0**	48.3	39.9	39.4	**64.7**	61.3	64.7	**36.3**	**57.0**	**40.2**	60.6	55.5	52.1	**50.7**	**26.5**	53.8	53.5	59.3	53.8
[16]+S-NMS L	**51.5**	**80.0**	53.2	55.8	**48.9**	**40.0**	**39.6**	64.6	**61.5**	**65.0**	**36.3**	56.5	**40.2**	**61.3**	**55.6**	**52.9**	50.3	26.2	**54.3**	**53.6**	**59.5**	**53.9**

Table 2. Results on Pascal VOC 2007 test set for off-the-shelf standard object detectors which use NMS as baseline and our proposed Soft-NMS method. Note that COCO-style evaluation is used.

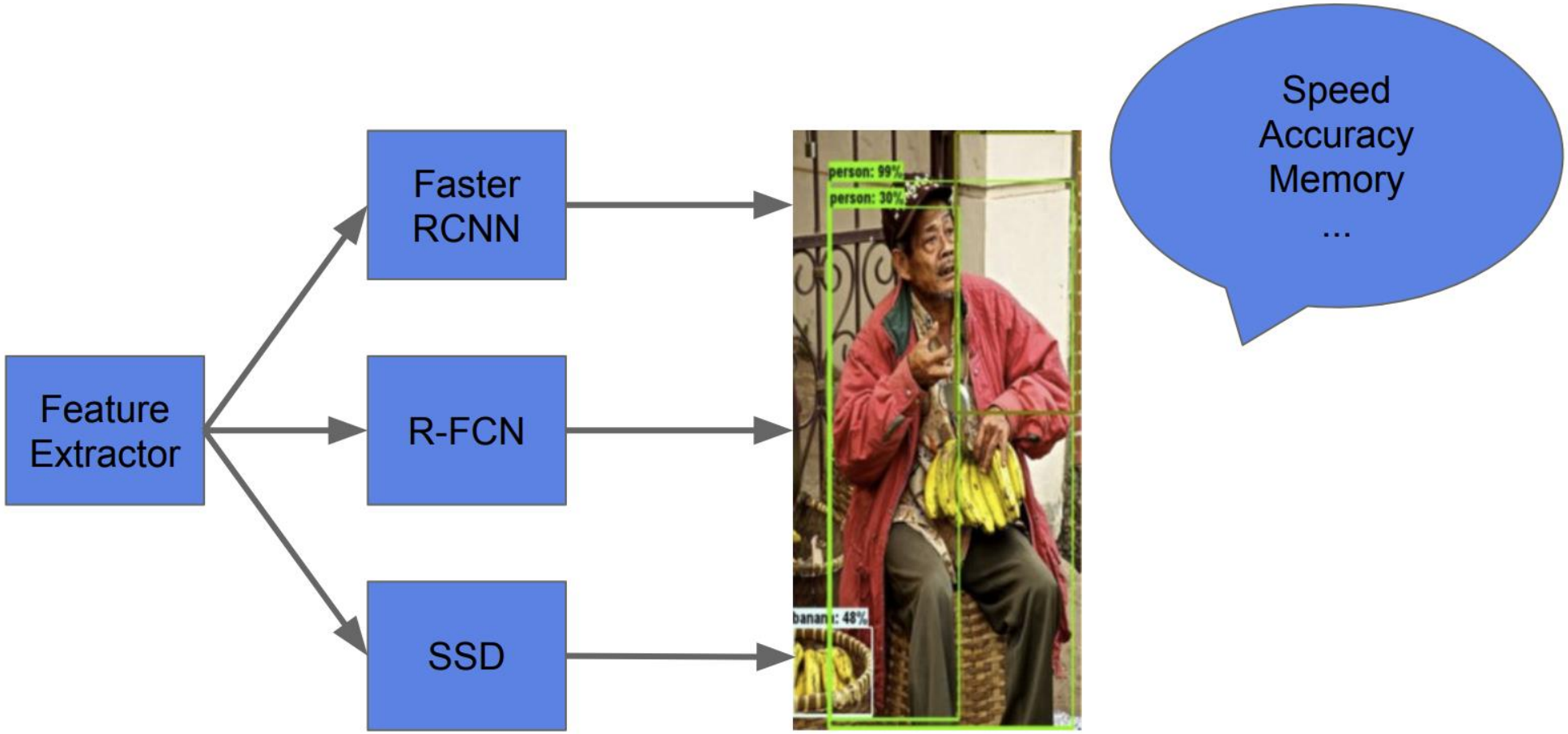
Compare deep neural network based object detectors

Speed / Accuracy Trade-off

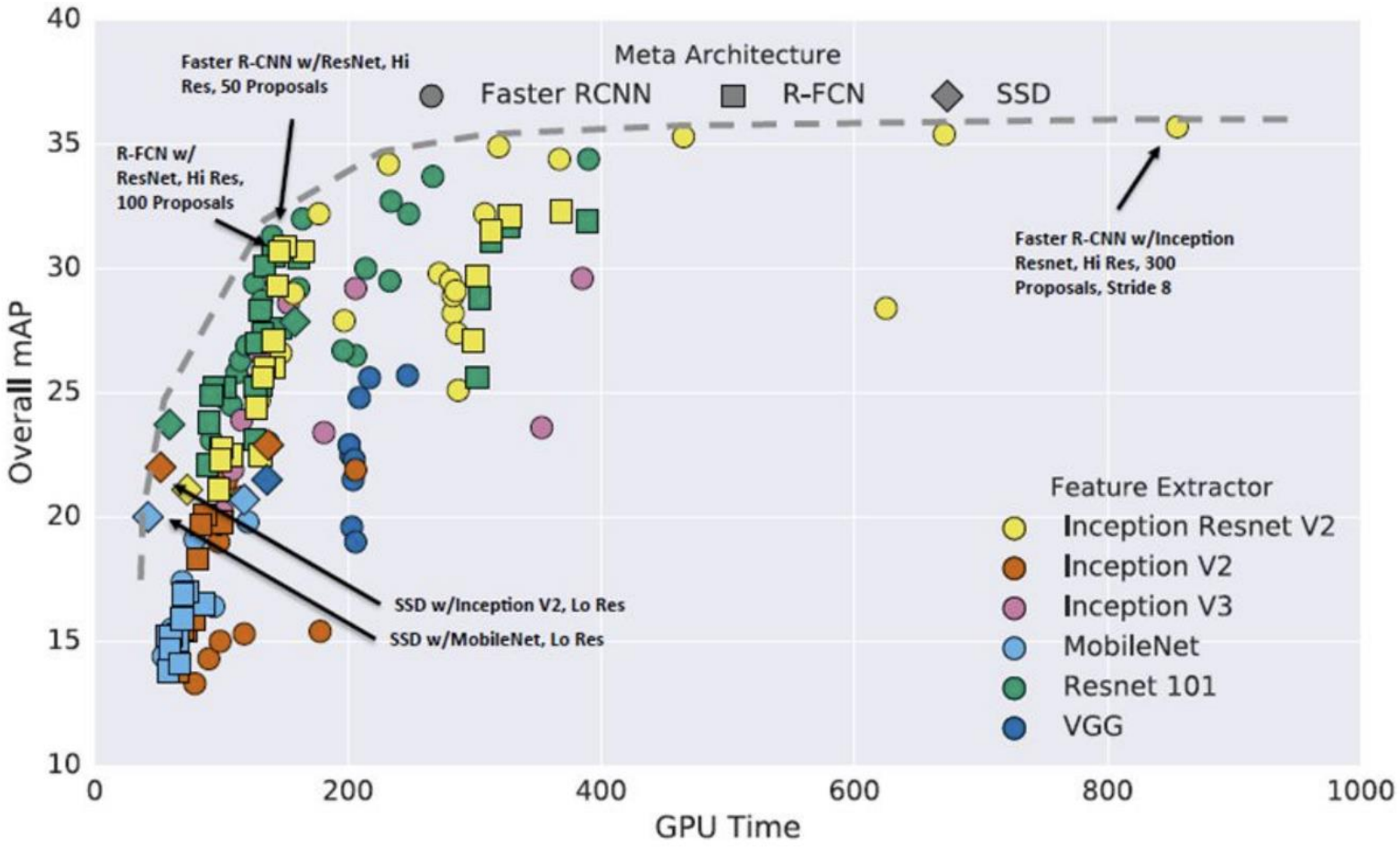
- Using unified Tensorflow architecture
- Compare speed, accuracy and memory usage

Huang, Murphy et al., "**Speed/Accuracy trade-offs for modern convolutional object detectors**", CVPR 2017

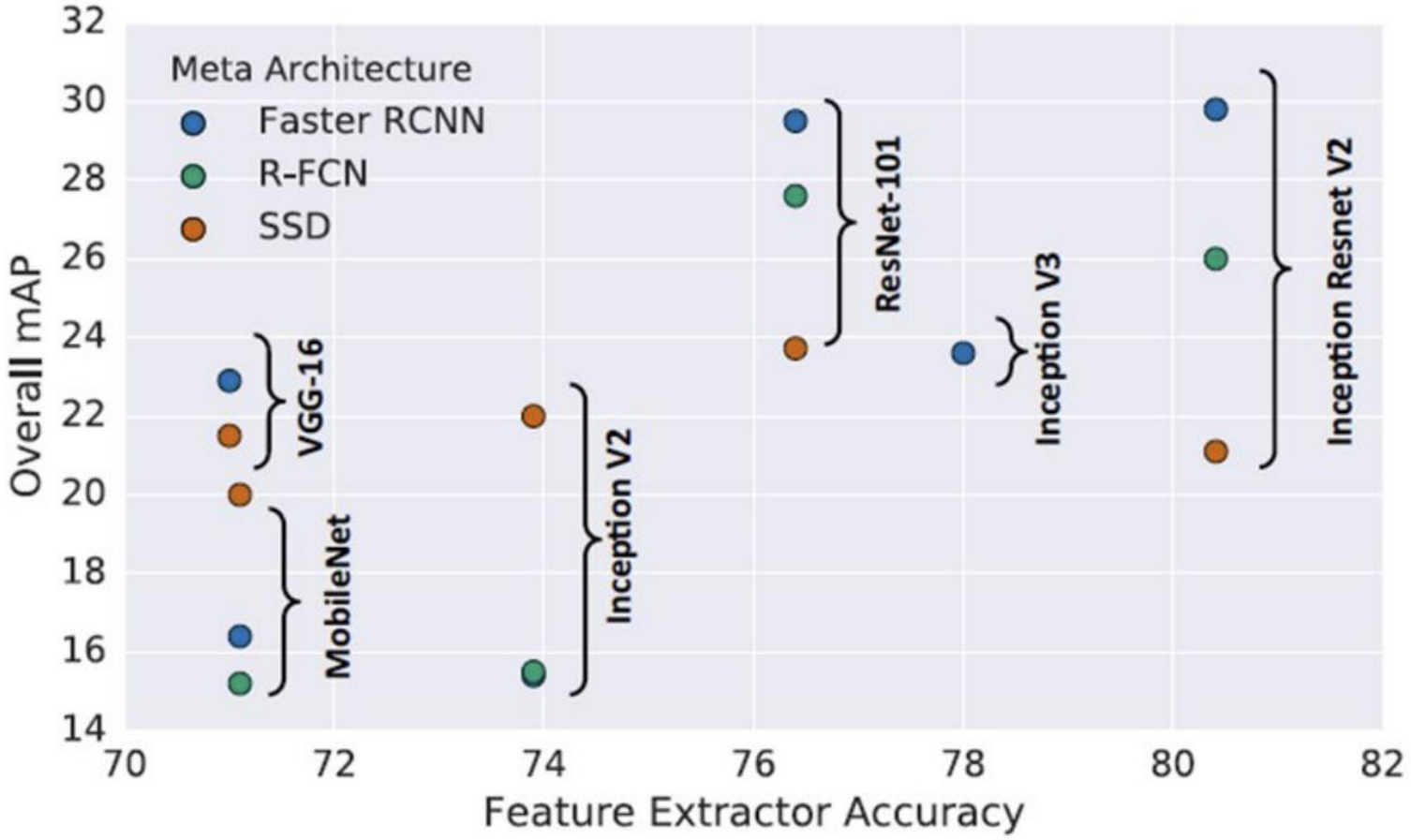
Same Architectures



Accuracy **vs** Speed



1. Different Feature Extractor



2. Detect Object Size

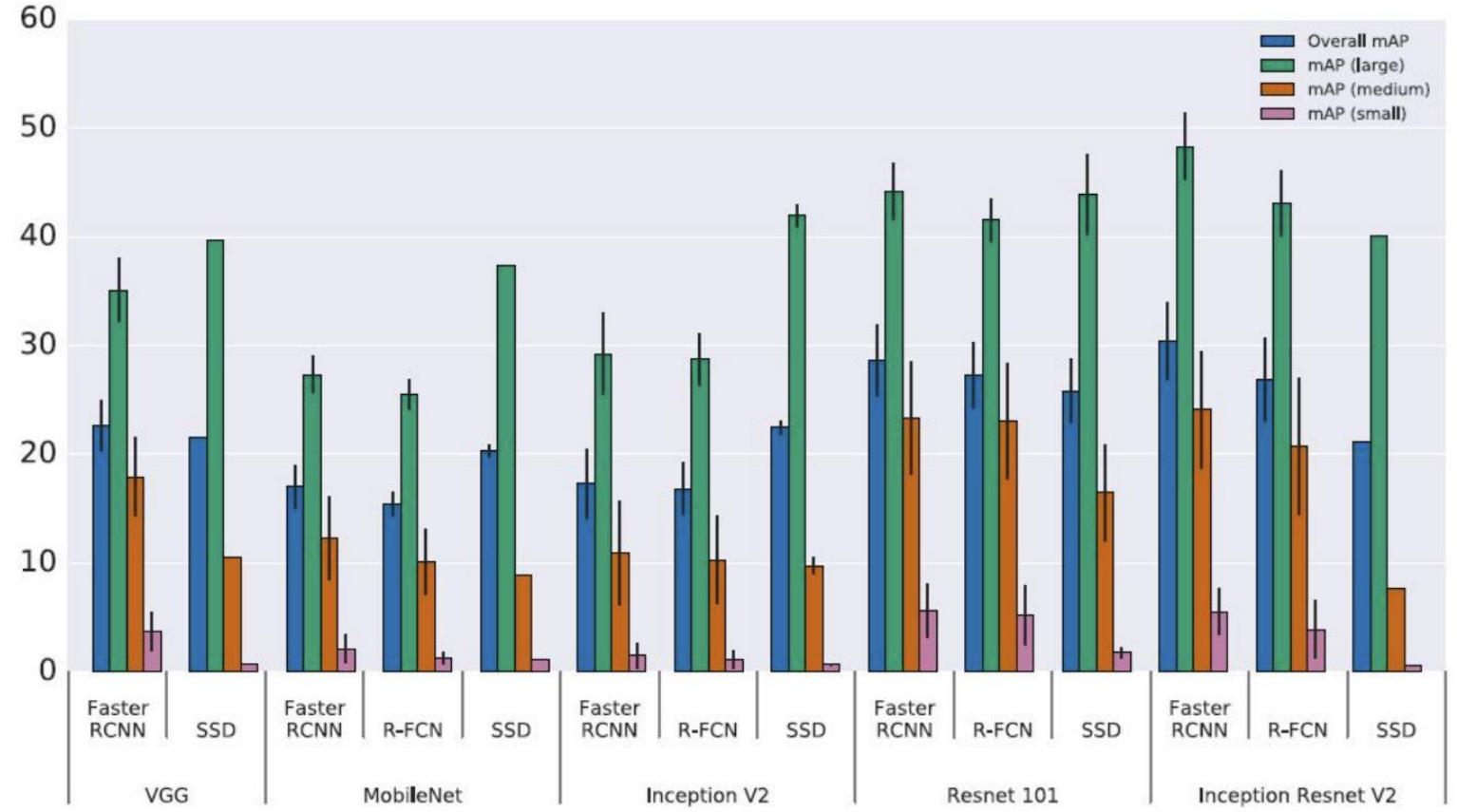
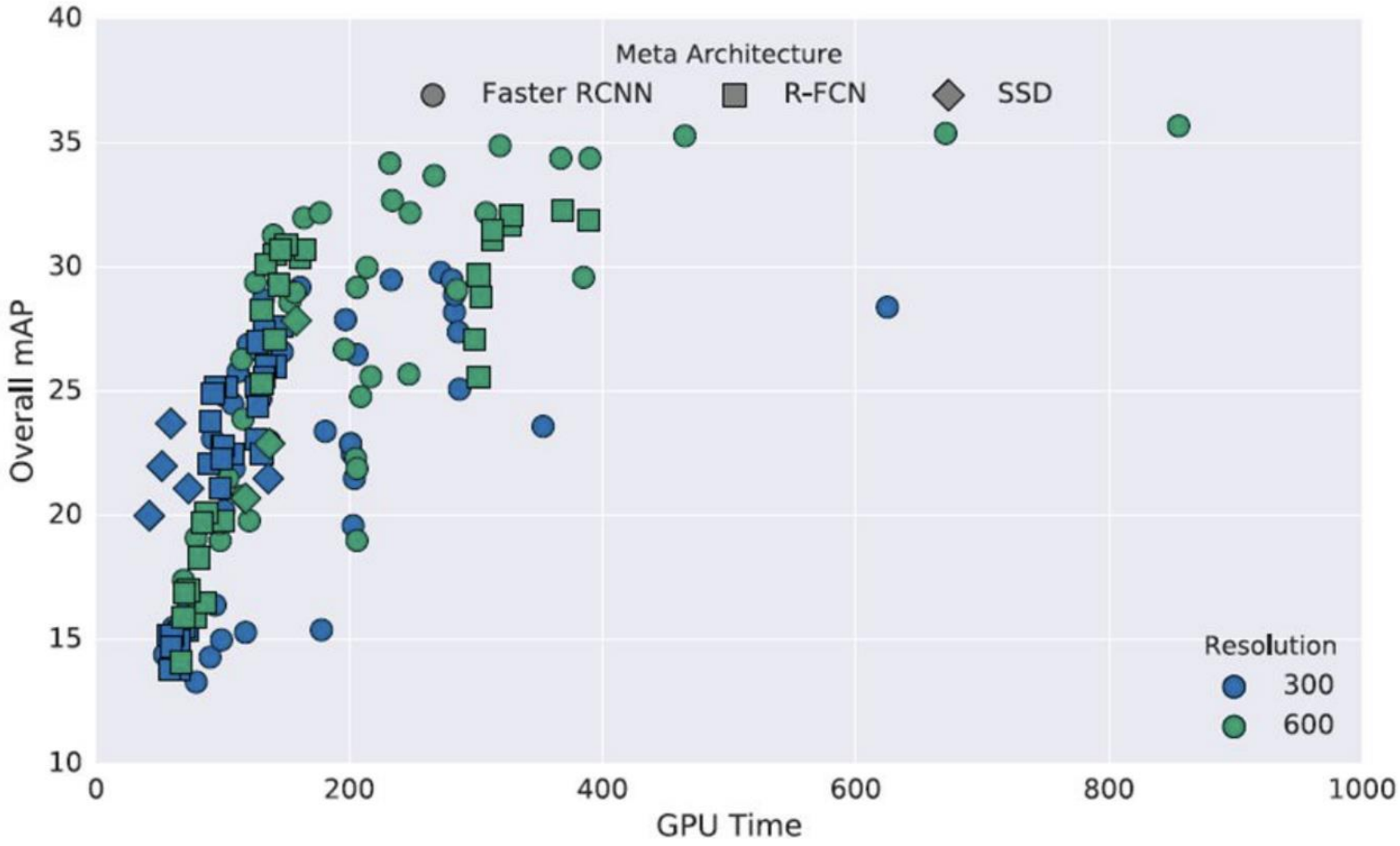
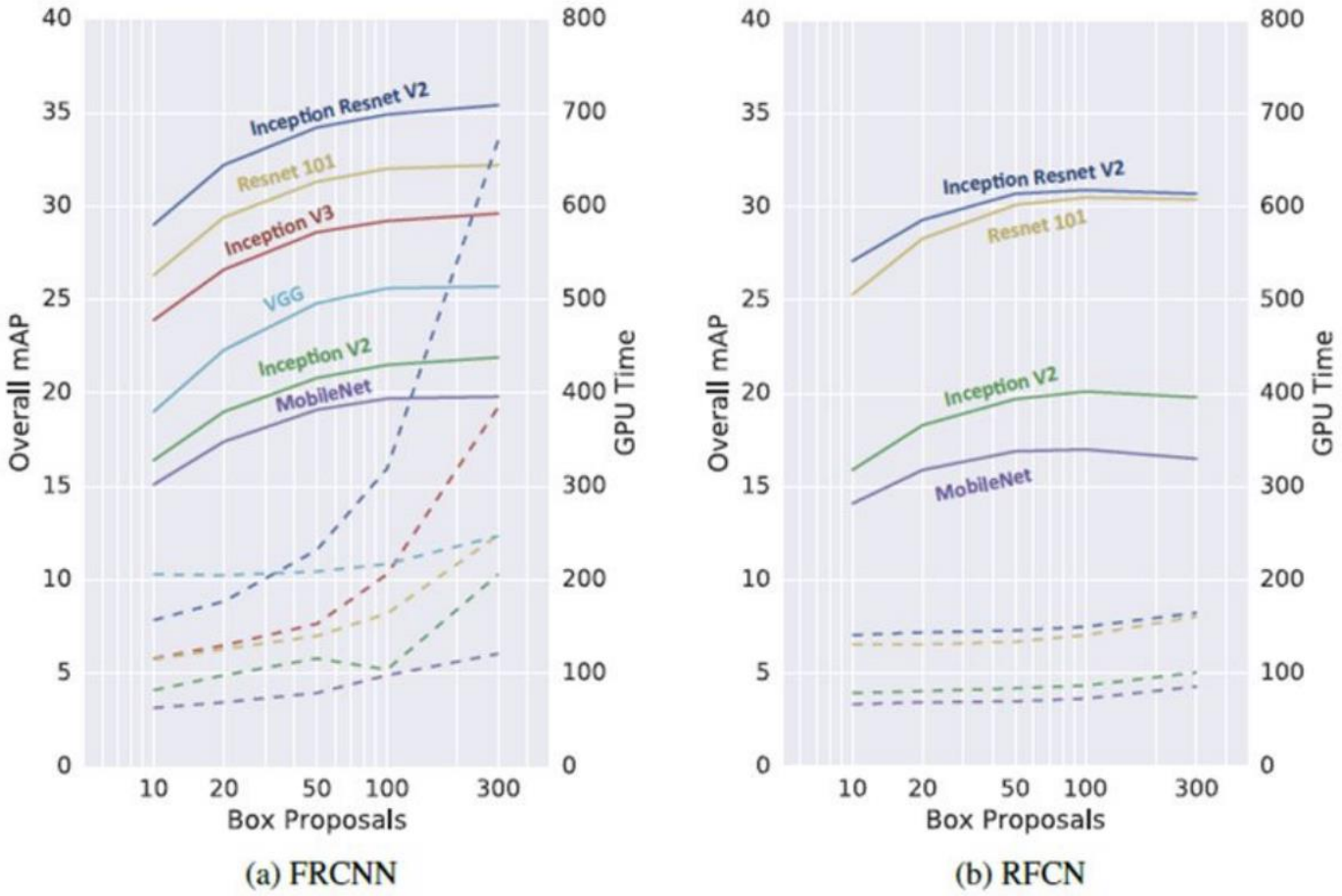


Figure 4: Accuracy stratified by object size, meta-architecture and feature extractor. We fix the image resolution to 300.

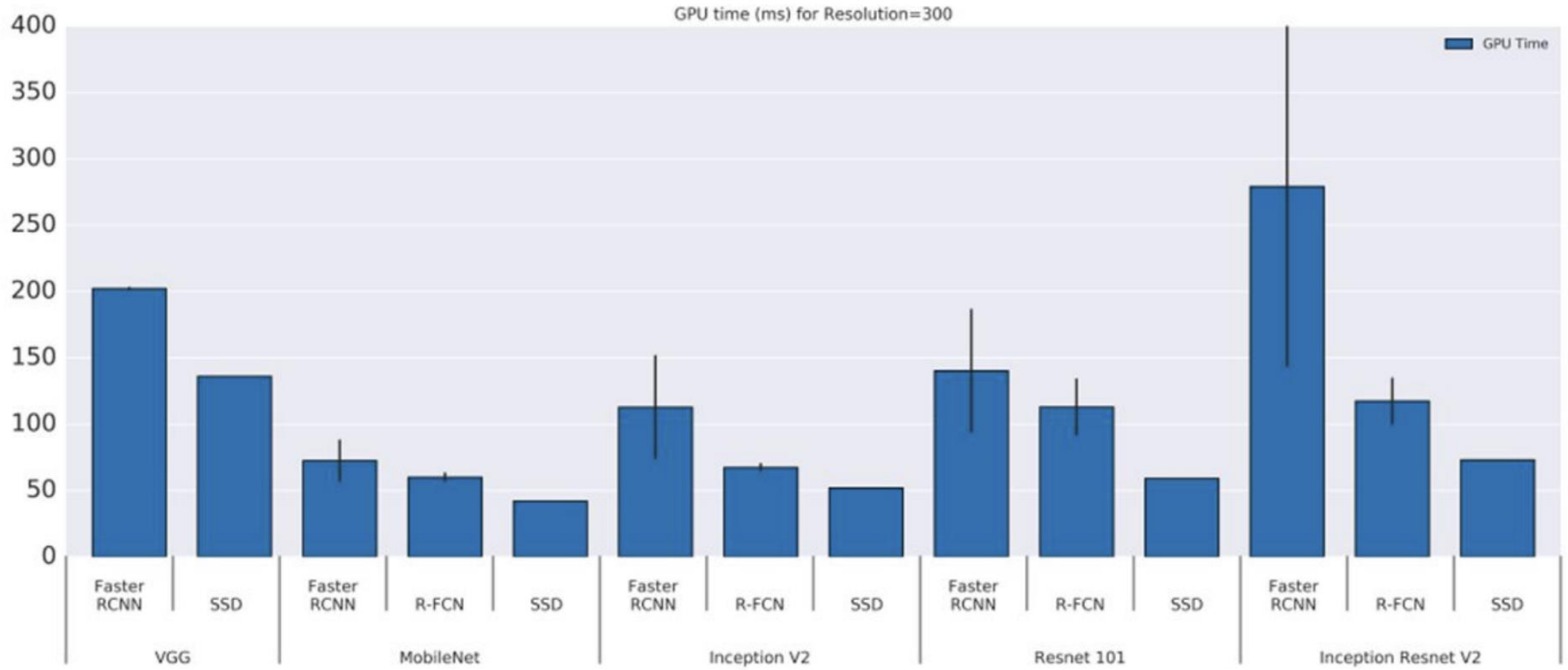
3. Image Resolution



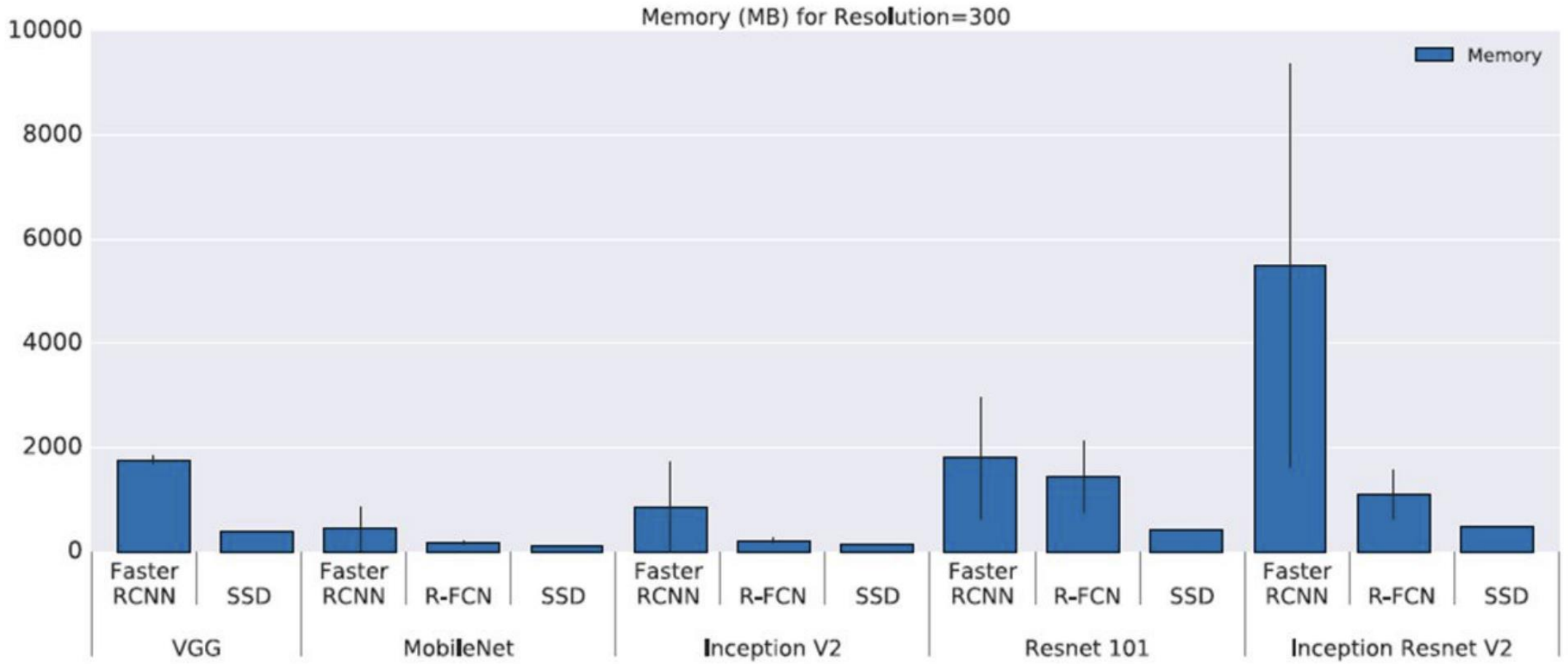
4. Number of Region Proposals



5. GPU Time


GPU time (ms) for Resolution=300
GPU Time
400
350
300
250
200
150
100
50
0
Faster RCNN
SSD
Faster RCNN
R-FCN
SSD
Faster RCNN
R-FCN
SSD
Faster RCNN
R-FCN
SSD
Faster RCNN
R-FCN
SSD
VGG
MobileNet
Inception V2
Resnet 101
Inception Resnet V2

6. Memory

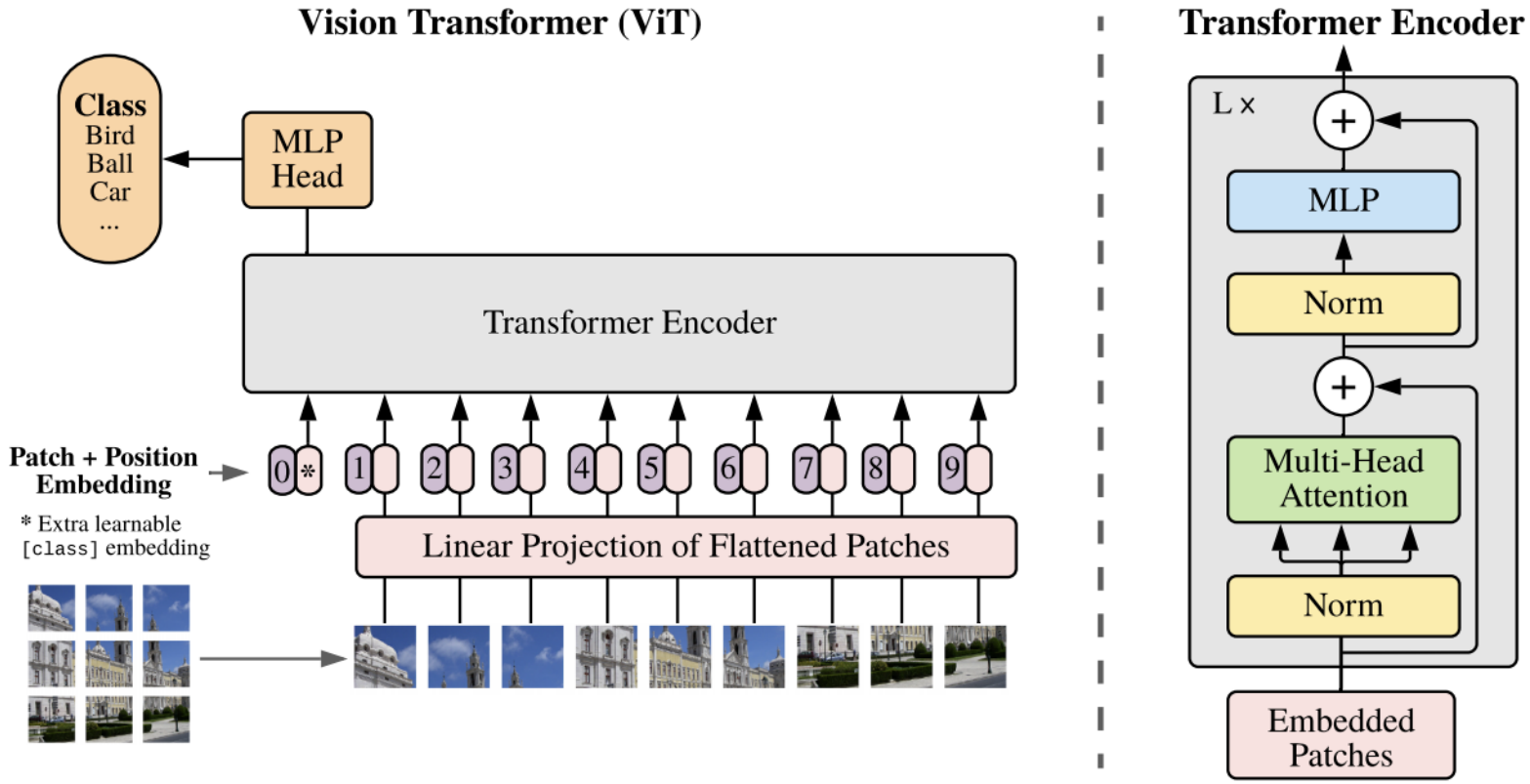

Memory (MB) for Resolution=300
Memory
10000
8000
6000
4000
2000
0
Faster RCNN
SSD
Faster RCNN
R-FCN
SSD
Faster RCNN
R-FCN
SSD
Faster RCNN
R-FCN
SSD
Faster RCNN
R-FCN
SSD
VGG
MobileNet
Inception V2
Resnet 101
Inception Resnet V2

Which model should we use?

- **Speed First**: SSD
- **Balance Speed and Accuracy**: R-FCN
- **Accuracy First**: Faster RCNN
  - Reduce proposals, it can speed up a lot with some accuracy loss

What about vision counter part?

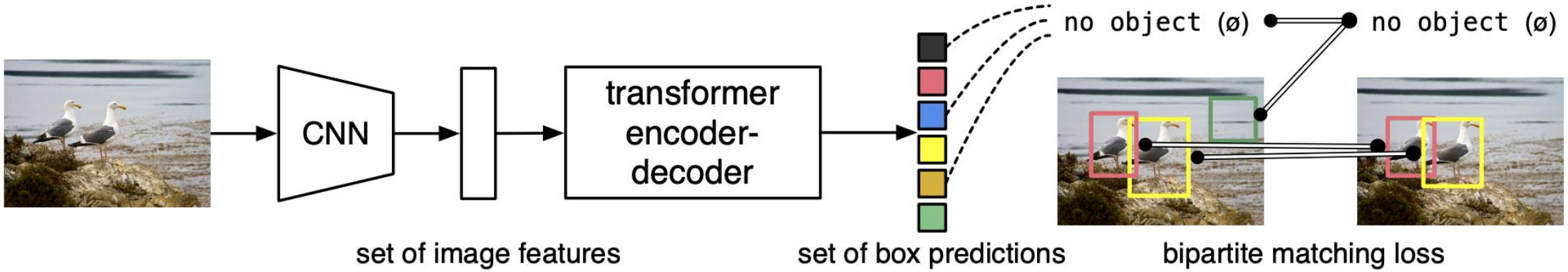
- Vision Transformer (ViT)



Dosovitskiy et al., "**AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE**", *ICLR 2021*

State of the art: DETR (**DE**tection **TR**ansformer)

- Using transformer for detection



Carion et. al., "**End-to-End Object Detection with Transformers**" *ECCV 2020*

DETR details

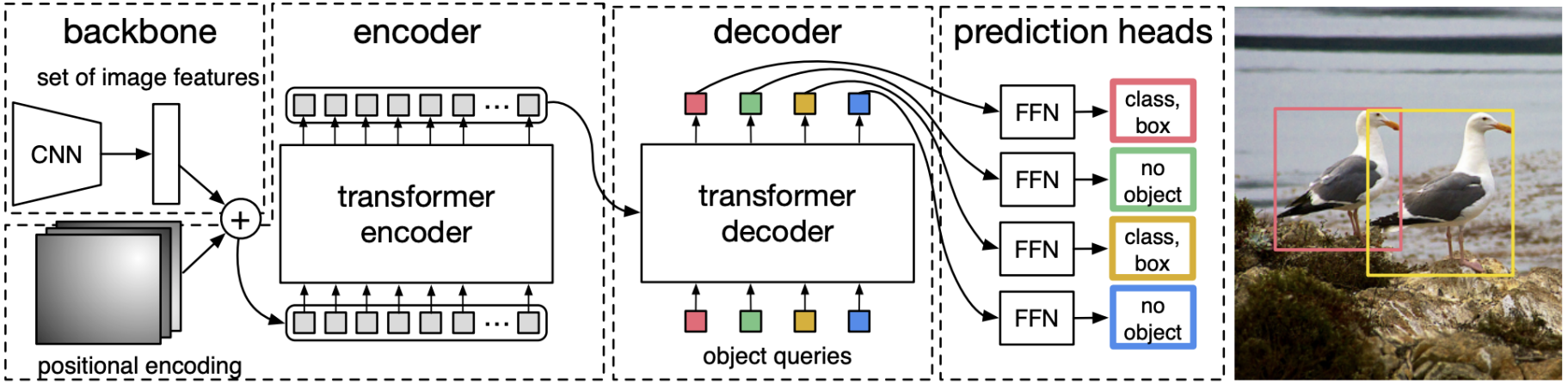


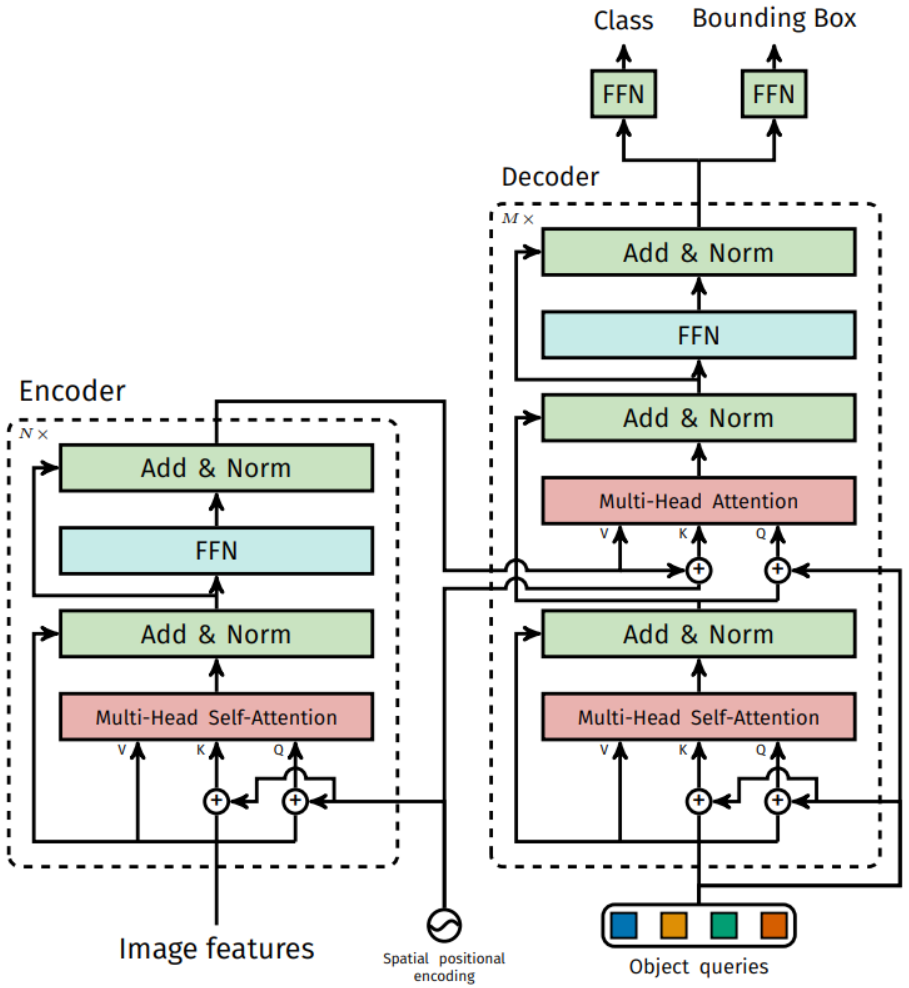
Fig. 2: DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call *object queries*, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a "`no object`" class.

Carion et. al., "**End-to-End Object Detection with Transformers**" *ECCV 2020*

DETR removes ~~annoying~~ NMS

- Using self-attention, each object queries will output
  - One class label
  - One bounding box
    - No NMS is required!

- **Additional benefit**: DETR uses ResNet and standard transformer architecture
  - Immediate use of existing deep learning library's implementations of those



DETR
Performance

- Using FPN, DETR may be improved!

Table 1: Comparison with Faster R-CNN with a ResNet-50 and ResNet-101 backbones on the COCO validation set. The top section shows results for Faster R-CNN models in Detectron2 [50], the middle section shows results for Faster R-CNN models with GIoU [38], random crops train-time augmentation, and the long **9x** training schedule. DETR models achieve comparable results to heavily tuned Faster R-CNN baselines, having lower AP_S but greatly improved AP_L. We use torchscript Faster R-CNN and DETR models to measure FLOPS and FPS. Results without R101 in the name correspond to ResNet-50.

Model	GFLOPS/FPS	#params	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Faster RCNN-DC5	320/16	166M	39.0	60.5	42.3	21.4	43.5	52.5
Faster RCNN-FPN	180/26	42M	40.2	61.0	43.8	24.2	43.5	52.0
Faster RCNN-R101-FPN	246/20	60M	42.0	62.5	45.9	25.2	45.6	54.6
Faster RCNN-DC5+	320/16	166M	41.1	61.4	44.3	22.9	45.9	55.0
Faster RCNN-FPN+	180/26	42M	42.0	62.1	45.5	26.6	45.4	53.4
Faster RCNN-R101-FPN+	246/20	60M	44.0	63.9	**47.8**	**27.2**	48.1	56.0
DETR	86/28	41M	42.0	62.4	44.2	20.5	45.8	61.1
DETR-DC5	187/12	41M	43.3	63.1	45.9	22.5	47.3	61.1
DETR-R101	152/20	60M	43.5	63.8	46.4	21.9	48.0	61.8
DETR-DC5-R101	253/10	60M	**44.9**	**64.7**	47.7	23.7	**49.5**	**62.3**

Summary

- Two stage detectors
  - R-CNN
  - Fast, Faster R-CNN
- Single stage detectors
  - YOLO
  - SSD
- Transformer based detectors
  - DETR